



A universal approach for solving the multi-revolution Lambert's problem

James McElreath¹ · Ian M. Down¹ · Manoranjan Majji¹

Received: 21 March 2025 / Revised: 27 May 2025 / Accepted: 3 June 2025
© The Author(s) 2025

Abstract

Lambert's problem, a cornerstone of orbital mechanics, is crucial in the design and planning of space missions, from satellite rendezvous to deep-space trajectory optimization. This paper presents a universal approach (valid across all conics) to solving Lambert's problem. By employing matrix exponential solutions to the Sundman transform, the transfer time equation—constructed of universal functions—is evaluated through exponential functions, resulting in two key advantages. The first advantage is the removal of all trigonometric functions and associated inverses when evaluating the transfer time function, reducing computation cost. The second is a unified transfer time formulation across elliptic and hyperbolic regimes, reducing algorithm complexity. Furthermore, the multi-revolution term is removed from the transfer time equation, reducing the complexity of transfer time derivatives and improving the numerical stability of multi-revolution solutions. A simple-but-accurate initial guess scheme that leverages the transfer angle is employed, resulting in accurate initial guesses and solutions even at 100,000 revolutions. Simulations indicate computation rates similar to state-of-the-art methods. Tests across a wide range of representative cases confirm that the solver achieves 10-digit velocity accuracy within two iterations for most transfer geometries and one iteration for near-circular cases, regardless of revolution count.

Keywords Lambert's problem · Orbital rendezvous · Lambert solver · Sundman transform · Gooding's method

Nomenclature

a	Semimajor axis
c	Chord length between $\mathbf{r}_0$ and $\mathbf{r}_f$
e	Orbit eccentricity

Ian M. Down and Manoranjan Majji have contributed equally to this work.

✉ James McElreath
j.mcelreath@tamu.edu

Ian M. Down
idown@tamu.edu

Manoranjan Majji
mmajji@tamu.edu

¹ Department of Aerospace Engineering, Texas A&M University, 710 Ross St, College Station, TX 77843, USA

E	Eccentric anomaly
f	True anomaly
H	Hyperbolic anomaly
$\hat{\mathbf{i}}_r, \hat{\mathbf{i}}_c$	Unit vectors in radial and chord directions
j	Imaginary unit ($j^2 = -1$)
m	Geometry-dependent constant used in transfer time equation
N	Number of revolutions
p	Semi-latus rectum
$\mathbf{r}_0, \mathbf{r}_f$	Initial and final position vectors
r_e	Earth equatorial radius
σ	Dot product of $\mathbf{r}$ and $\mathbf{v}$ over $\sqrt{\mu}$
t	Time
T	Scaled transfer time function
T', T'', T'''	First, second, and third derivative of T with respect to x
T_0, T_θ, T_m	Transfer times for parabolic, transfer angle-based, and minimum time solutions
$\tilde{T}$	Scaled input transfer time
θ	Transfer angle
$\tilde{\theta}$	$2\pi - \theta$
U_n, U_n^*	Universal functions
$\mathbf{v}_0, \mathbf{v}_f$	Initial and final velocity vectors
v_c, v_ρ	Chordal and radial components of velocity
x	Universal variable ($\sqrt{\alpha}\chi$)
x_0, x_m, x_τ	Initial guess, minimum time, and zero-crossing values of x
α	Inverse of the semimajor axis ($1/a$)
$\alpha', \alpha'', \alpha'''$	First, second, and third derivative of α with respect to x
χ	Sundman-transformed independent variable
Δt	Time of flight or transfer time
$\Delta \tilde{t}$	Input transfer time
ϵ	Iteration convergence threshold
η	Transfer direction parameter, $\eta = 0$ for short-way and $\eta = 1$ for long way
μ	Gravitational parameter

1 Introduction

Lambert's problem has long been recognized as a fundamental problem in astrodynamics, forming the cornerstone of trajectory design, mission planning, and spacecraft rendezvous in the two-body problem. Formulated by Johann Heinrich Lambert in the eighteenth century, the problem aims to determine the connecting orbit between two position vectors from a given time of flight, yielding the orbital velocity at the two positions. Originally used to determine the orbits of planets and other celestial bodies, Lambert's problem has proven particularly useful when determining rendezvous maneuvers between two orbiters. In practice, the solution can be extended to allow multiple revolutions between the position vectors, commonly referred to as the multi-revolution Lambert problem, with cases involving hundreds of revolutions sometimes termed ultra-revolution scenarios. In this paper, the authors present a refined approach to solving Lambert's problem that preserves the speed and accu-

racy of modern methods while offering a clear, approachable implementation, even at large revolution counts.

Many methods to solve Lambert's problem have been developed over the years, with most typically solving through iterative methods to refine an initial guess. Lagrange's contribution to Lambert's problem, specifically his formulation using elliptic integrals, laid the foundation for much of modern astrodynamics by making it possible to calculate orbital trajectories based on position and time. Gauss's successive substitution method improved computational feasibility but struggled with convergence, particularly for large transfer angles or hyperbolic trajectories (Battin 1999). The inefficiencies in Gauss's approach were addressed by subsequent researchers like Battin and Vaughan (1984), who proposed a more robust method that enhanced convergence by exploiting the invariance of the mean point under a certain geometric transformation of the boundary-value problem. Lancaster and Blanchard (1969) made a significant advancement in solving Lambert's problem by proposing a unified formulation that could be applied to elliptic, hyperbolic, and parabolic orbits through a universal variable. This unification allowed Lambert solvers to be used more broadly and offered computational efficiency across various orbital regimes, which proved instrumental for space missions involving complex orbital maneuvers. Around the same time, Bate et al. (2020) developed their universal approach, utilizing a different universal variable related to the transfer anomaly. Escobal (1965) contributed foundational knowledge on orbit determination, formalizing many of the key concepts underlying modern Lambert solvers. Boltz (1984) further refined the mathematical underpinnings of Lambert solvers with his iterative solution to the Lambert/Gauss problem, enhancing both efficiency and applicability to a wider range of orbital problems. The contribution of Gooding (1990) marked a significant milestone when he developed a highly efficient and robust algorithm with rapid convergence for both elliptic and hyperbolic orbits. Built upon Lancaster and Blanchard's work, Gooding's method remains one of the fastest and most accurate solvers to date and is frequently used in astrodynamics where speed and precision are paramount. Additionally, the comprehensive treatise of Vallado (2001) on astrodynamics has been instrumental in disseminating practical methods for orbit determination, including the application of Lambert solvers in real-world missions. His work has become a standard reference for both theoretical and practical aspects of astrodynamics, offering detailed algorithms for implementing Lambert solvers.

Modern advancements in Lambert solvers have focused on improving computational efficiency and robustly handling "stress cases" (see Table 1 of Russell 2019). Contributions have been made by Shen and Tsiotras (2003), who expanded Battin's approach to handle multi-revolution cases efficiently. Their research is valuable for long-duration missions and planetary flybys where spacecraft may complete multiple orbits before reaching their target. The work of Izzo (2015), an extension of Lancaster and Blanchard's method, made significant strides by introducing a solver optimized for high-speed, large-scale computations. Employing a logarithmic variable transformation and a simple-but-accurate initial guess scheme, Izzo showed significant speed improvements over Gooding's method (20–30%). However, Izzo's method has been seen to be less robust when compared to others (Russell 2022), as it approximates the minimum transfer time when solving multi-revolution solutions, resulting in cases where the solver takes longer to converge or converges to an incorrect solution. Arora and Russell (2013), motivated by the universal variable formulation of Bate et al. (2020), developed a vercosine transformation to simplify multi-revolution Lambert problems. The runtime of the solver compared favorably to Gooding (faster by 40–55%), mainly through the use of sophisticated initial guesses, a high-order root solver, and a formulation with fewer inverse trigonometric functions. Russell (2019) furthered this work by exploiting the fundamental 2D nature of the parameter space to fully interpolate the solution using the

vercosine formulation. Runtime improvements of 38–68% were reported when compared to Gooding's, but required large coefficient files (~50 megabytes for each number of revolutions) for the initial guess scheme. Russell (2022) was able to reduce this memory footprint substantially (~1 megabyte total) in a follow-up work, accomplished through a new variable transformation that made the 2D surface for each revolution case almost invariant to the number of revolutions. Finally, an exact solution to Lambert's problem, developed by Negrete and Abdelkhalik (2025), was derived in terms of a ratio of two complex contour integrals for zero-revolution solutions and as the roots of a quadratic in terms of contour integrals for N-revolution transfers. The composite trapezoidal rule was used to approximate these integrals, with the accuracy and computation speed tied to the number of points used in the approximation. The method showed favorable computation speeds while maintaining accuracy when the proper number of points was selected.

To the best of the authors' knowledge, the most widely used Lambert solvers today are those of Battin (1999); Gooding (1990); Izzo (2015); Arora and Russell (2013), and Russell (2019, 2022), with perhaps Gooding being the most widely implemented. Battin's method is extremely robust and does not require different transfer time evaluation implementations between elliptic and hyperbolic cases. However, it is generally slower than Gooding's (Klumpp 1991; Peterson et al. 2010), and its use of continued fractions can also be unintuitive for practitioners. Gooding's method is also considered robust, containing an accurate initial guess scheme and fast convergence. Even so, it is outmatched in terms of speed when compared to the newer methods of Izzo and Russell. The implementation methodology in the original report is also difficult to decipher and its initial guess scheme degrades at high numbers of revolutions. Izzo's method is perhaps the simplest in terms of implementation, equations, and initial guess scheme, but has been seen to be less robust than that of Gooding's method. Most notably, Izzo's method shows poor performance when single-revolution case flight times are excessively high and for multi-revolution cases with huge N revolutions (Russell 2022). Russell's method is the fastest of these four methods while retaining the robust solving nature, but it is perhaps the most sophisticated algorithm with its interpolation scheme, required memory files, and a vercosine transformation. This work aims to unify the advantages of these methods while streamlining some of the implementation artifacts. In particular, the new solver is designed to be robust and simple to implement, retain accurate initial guesses and solutions regardless of revolution count, and match the solve speeds of state-of-the-art methods. Such a solver meets the needs of modern mission design, ensuring consistency, efficiency, and ease of adoption for practitioners.

Existing solvers all contain at least one of the following aspects: different implementations of transfer time evaluation between elliptic and hyperbolic regimes, use of base/hyperbolic trigonometric functions and/or their inverses each time the transfer time is computed, poor performance for large numbers of revolutions, and/or sophisticated algorithm implementation and/or complex initial guess schemes. In this work, a method to solve Lambert's problem is derived under these considerations. Building upon the work of Bate et al. (2020), the presented method makes use of the Sundman (1913) transform and its matrix exponential solution to evaluate a transfer time equation that is comprised of universal functions. Universal functions, determined by solving the linear time-invariant differential equations of the Sundman transform (Battin 1999), are well-documented functions that are "universal" across conics and are used for classical equations like Lagrange coefficients and Kepler's equation. Put simply, universal functions allow a "one-size-fits-all" approach to conic orbit computations, saving effort and minimizing errors when transitioning between different orbit regimes. The true advantage of these universal functions is that, as shown by Majji (2020), they can be evaluated through the use of exponential functions, resulting in the removal of

base and inverse trigonometric functions from the Lambert solving iteration process. The removal of the trigonometric functions also removes the need for different implementations of transfer time evaluations between elliptic and hyperbolic regimes, reducing implementation complexity. To evaluate the universal functions, a universal variable that is related to the transfer angle is employed. Its relation to the transfer angle makes it intuitive to understand and allows for the variable to be easily bounded. A simple-but-accurate initial guess scheme is also employed, leveraging the transfer angle due to its relation to the universal variable utilized in this work. Because the universal variable is bounded, the initial guesses for large multi-revolution solutions retain their accuracy regardless of the revolution number. Multi-revolution cases are also handled by properly selecting the bounds on the universal variable. This removes the need for a supplementary multi-revolution term in the transfer time equation, resulting in reduced transfer time partial derivatives and improved numerical stability.

This paper is structured as follows. First, the Sundman transform is reviewed and it is shown how the matrix exponential solution yields equations for the universal functions independent of the conic type. Second, the new Lambert solver is introduced and it is shown how to evaluate the transfer time equation from the universal variable and universal functions. Third, the initial guess schemes, root-solving methods, and velocity reconstruction are discussed and an algorithm summarizing the solver method is presented. Finally, numerical simulations are presented that compare the proposed method against Gooding's method over a wide variety of representative cases.

2 Sundman transform and universal functions

While the traditional methods for solving Lambert's problem have proven effective, a method to solve while addressing five distinct aspects of the problem is desired. These aspects include:

1. Equivalent implementations of transfer time evaluation between elliptic and hyperbolic regimes.
2. The removal of base/hyperbolic trigonometric functions and/or their inverses during the iteration process.
3. Straightforward algorithm implementation.
4. Simple-but-accurate initial guess schemes.
5. The retention of accurate initial guesses and solutions for large numbers of revolutions.

The Sundman transform in conjunction with matrix exponential solutions is employed to address the first two aspects, while the other three aspects are addressed when formulating the Lambert solver (Section 3). The Sundman transform reparameterizes time, simplifying the two-body problem, while the matrix exponential provides an elegant and computationally efficient means to determine the universal functions. The universal functions are used to construct the transfer time equation, requiring only one trigonometric function evaluation and removing the need for separate algorithmic implementations between elliptic and hyperbolic regimes.

2.1 The Sundman transform

Consider a particle in a central force field governed by Newton's law of gravitation. The standard equation of motion is given by:

$$\ddot{\mathbf{r}} = \frac{d^2\mathbf{r}}{dt^2} = -\frac{\mu}{||\mathbf{r}||^3}\mathbf{r} \quad (1)$$

where $\mathbf{r}$ is the position vector of the orbiter and μ is the gravitational parameter of the central body. In the Sundman transform, the time variable t is reparameterized as a new variable, χ , defined as:

$$r d\chi = \sqrt{\mu} dt \quad (2)$$

where $r = \|\mathbf{r}\|$. This substitution transforms the original nonlinear differential equations into new differential equations that are linear in χ . The key idea here is that the Sundman transform introduces a new parameterization, effectively regularizing the problem by removing singularities associated with the original nonlinear dynamics, such as transfer angles of 180° or trajectories near the central body (small values of r). After applying the Sundman transform, the position vector's first, second, and third derivatives with respect to χ are written as:

$$\frac{d\mathbf{r}}{d\chi} = \frac{r}{\sqrt{\mu}} \mathbf{v} ; \quad \frac{d^2\mathbf{r}}{d\chi^2} = \frac{\sigma}{\sqrt{\mu}} \mathbf{v} - \frac{\mathbf{r}}{r} ; \quad \frac{d^3\mathbf{r}}{d\chi^3} = -\alpha \frac{d\mathbf{r}}{d\chi} \quad (3)$$

where $\mathbf{v}$ is the orbiter velocity, α is equal to the inverse of the semimajor axis (a)

$$\alpha = \frac{1}{a} \quad (4)$$

and

$$\sigma = \frac{\mathbf{r} \cdot \mathbf{v}}{\sqrt{\mu}} \quad (5)$$

Taking these equations, combining, and then integrating from an initial time t_0 to a final time t_f yields:

$$\chi = \alpha \sqrt{\mu} (t_f - t_0) + \sigma_f - \sigma_0 \quad (6)$$

where

$$\sigma_0 = \sigma(t_0) \quad ; \quad \sigma_f = \sigma(t_f) \quad (7)$$

Equation 6 is the universal Kepler's equation and will be particularly useful when forming the transfer time equation through universal functions.

The Sundman transform plays a critical role in addressing some of the challenges posed by traditional formulations of Lambert's problem. However, the true strength of this approach is revealed when combined with efficient numerical techniques such as the matrix exponential method. The following section delves into how these two methodologies—the Sundman transform and matrix exponential—work together to streamline the solution process for Lambert's problem, offering a robust framework applicable across a variety of orbital regimes.

2.2 Evaluating the universal functions

Beyond the advantages of regularizing the dynamics using the Sundman transform, Majji (2020) introduced the matrix exponential approach to efficiently propagate an orbiter operating under the two-body assumption. This method streamlines the computation of universal functions, making the overall solution process more computationally feasible for modern

astrodynamics applications. The state-space formulation of the regularized two-body problem can be written in the form of a first-order linear differential equation:

$$\frac{d}{d\chi} \mathbf{z} = \frac{d}{d\chi} \begin{bmatrix} \mathbf{r} \\ \frac{d\mathbf{r}}{d\chi} \\ \frac{d^2\mathbf{r}}{d\chi^2} \end{bmatrix} = \mathbf{A} \begin{bmatrix} \mathbf{r} \\ \frac{d\mathbf{r}}{d\chi} \\ \frac{d^2\mathbf{r}}{d\chi^2} \end{bmatrix} = \mathbf{A}\mathbf{z} \quad (8)$$

where $\mathbf{A}$ is a block matrix representing the system dynamics, assembled by use of Eq. 3. The system matrix $\mathbf{A}$ has a specific block structure that arises from the linearization of the equations of motion. In the unperturbed case, $\mathbf{A}$ is given as:

$$\mathbf{A} = \begin{bmatrix} 0 & \mathbf{I} & 0 \\ 0 & 0 & \mathbf{I} \\ 0 & -\alpha\mathbf{I} & 0 \end{bmatrix} \quad (9)$$

where $\mathbf{I}$ is the identity matrix. The general solution to this system of linear time-invariant differential equations is given by the matrix exponential:

$$\mathbf{z}(\chi) = \exp(\mathbf{A}\chi)\mathbf{z}_0 \quad (10)$$

where $\mathbf{z}_0$ represents the vector at initial time (t_0) and $\exp(\mathbf{A}\chi)$ is the matrix exponential of the system matrix $\mathbf{A}$. The matrix exponential can be expanded as a power series:

$$\exp(\mathbf{A}\chi) = \mathbf{I} + \mathbf{A}\chi + \frac{\mathbf{A}^2\chi^2}{2!} + \frac{\mathbf{A}^3\chi^3}{3!} + \dots \quad (11)$$

Using the definition of universal functions (Battin 1999)

$$U_n(\chi; \alpha) = \chi^n \left[\frac{1}{n!} - \frac{\alpha\chi^2}{(n+2)!} + \frac{(\alpha\chi^2)^2}{(n+4)!} - \dots \right] \quad (12)$$

Majji (2020) identified that the matrix exponential can be expressed in terms of the universal functions as:

$$\exp(\mathbf{A}\chi) = \mathbf{I} + \mathbf{A}U_1(\chi, \alpha) + \mathbf{A}^2U_2(\chi, \alpha) = \begin{bmatrix} \mathbf{I} & U_1\mathbf{I} & U_2\mathbf{I} \\ 0 & U_0\mathbf{I} & U_1\mathbf{I} \\ 0 & -\alpha U_1\mathbf{I} & U_0\mathbf{I} \end{bmatrix} \quad (13)$$

To evaluate the matrix exponential function, consider the spectral decomposition of $\mathbf{A}$ written as:

$$\mathbf{A} = \begin{bmatrix} \mathbf{I} & \mathbf{I} & \mathbf{I} \\ 0 & j\sqrt{\alpha}\mathbf{I} & -j\sqrt{\alpha}\mathbf{I} \\ 0 & -\alpha\mathbf{I} & -\alpha\mathbf{I} \end{bmatrix} \begin{bmatrix} 0 & 0 & 0 \\ 0 & j\sqrt{\alpha}\mathbf{I} & 0 \\ 0 & 0 & -j\sqrt{\alpha}\mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{I} & 0 & \frac{1}{\alpha}\mathbf{I} \\ 0 & -j\frac{1}{2\sqrt{\alpha}}\mathbf{I} & -\frac{1}{2\alpha}\mathbf{I} \\ 0 & j\frac{1}{2\sqrt{\alpha}}\mathbf{I} & -\frac{1}{2\alpha}\mathbf{I} \end{bmatrix} \quad (14)$$

The spectral decomposition then enables the matrix exponential to be determined efficiently as:

$$\exp(\mathbf{A}\chi) = \begin{bmatrix} \mathbf{I} & \mathbf{I} & \mathbf{I} \\ 0 & j\sqrt{\alpha}\mathbf{I} & -j\sqrt{\alpha}\mathbf{I} \\ 0 & -\alpha\mathbf{I} & -\alpha\mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{I} & 0 & 0 \\ 0 & e^{j\sqrt{\alpha}\chi}\mathbf{I} & 0 \\ 0 & 0 & e^{-j\sqrt{\alpha}\chi}\mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{I} & 0 & \frac{1}{\alpha}\mathbf{I} \\ 0 & -j\frac{1}{2\sqrt{\alpha}}\mathbf{I} & -\frac{1}{2\alpha}\mathbf{I} \\ 0 & j\frac{1}{2\sqrt{\alpha}}\mathbf{I} & -\frac{1}{2\alpha}\mathbf{I} \end{bmatrix} \quad (15)$$

where j is the imaginary unit defined as $j^2 = -1$. By multiplying Eq. 15 through and utilizing the scheme given in Eq. 13, analytical expressions for the first four universal functions are

recoverable.

$$U_0(\chi; \alpha) = \frac{1}{2} \left(e^{j\sqrt{\alpha}\chi} + e^{-j\sqrt{\alpha}\chi} \right) \quad (16)$$

$$U_1(\chi; \alpha) = \frac{1}{2\sqrt{\alpha}} \left(-je^{j\sqrt{\alpha}\chi} + je^{-j\sqrt{\alpha}\chi} \right) \quad (17)$$

$$U_2(\chi; \alpha) = \frac{1}{2\alpha} \left(2 - e^{j\sqrt{\alpha}\chi} - e^{-j\sqrt{\alpha}\chi} \right) = \frac{1 - U_0}{\alpha} \quad (18)$$

$$U_3(\chi; \alpha) = \frac{1}{2\alpha^{3/2}} \left(2\chi\sqrt{\alpha} + je^{j\sqrt{\alpha}\chi} - je^{-j\sqrt{\alpha}\chi} \right) = \frac{\chi - U_1}{\alpha} \quad (19)$$

Note that the following relation also applies:

$$U_n + \alpha U_{n+2} = \frac{\chi^n}{n!} \quad (20)$$

Because they will be useful when assembling the transfer time equation, the first two universal functions of $\frac{1}{2}\chi$ are defined:

$$U_i^*(\chi; \alpha) = U_i\left(\frac{1}{2}\chi; \alpha\right) \quad (21)$$

or more specifically:

$$U_0^*(\chi; \alpha) = \frac{1}{2} \left(e^{\frac{1}{2}j\sqrt{\alpha}\chi} + e^{-\frac{1}{2}j\sqrt{\alpha}\chi} \right) \quad (22)$$

$$U_1^*(\chi; \alpha) = \frac{1}{2\sqrt{\alpha}} \left(-je^{\frac{1}{2}j\sqrt{\alpha}\chi} + je^{-\frac{1}{2}j\sqrt{\alpha}\chi} \right) \quad (23)$$

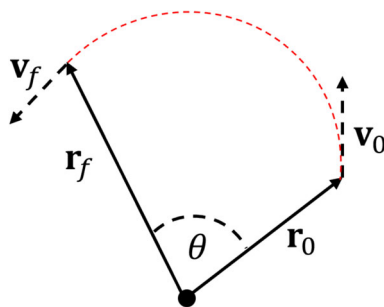
where the following identities apply:

$$U_2 = 2(U_1^*)^2 \quad (24)$$

$$U_1 = 2U_0^*U_1^* \quad (25)$$

Thus, the universal functions can be evaluated through exponential functions using standard mathematical libraries. As a result, the universal functions can be determined efficiently and without the need for separate equations for hyperbolic and elliptic trajectories. Linear systems theory provides well-established, efficient algorithms for evaluating exponential functions (Moler and Van Loan 1978), making exponential functions particularly attractive for use in Lambert's problem. Because e^x does not require the explicit argument reduction that trigonometric functions do, its implementation can be slightly simpler in many math libraries (Muller 2016; Muller et al. 2018). This often leads to e^x being slightly faster to compute than $\sin x$ or $\cos x$. However, some machines or programming languages are better suited for trigonometric function evaluations instead of exponential functions. When this is the case, expressions of the universal functions by trigonometric functions are desired, given in Appendix C. Note that in this formulation, only base trigonometric functions are needed to evaluate the universal functions, *so even if the trigonometric evaluations are used, there is no need to swap formulations between hyperbolic and elliptic regimes*. Since both evaluation types require four function evaluations, either four exponential functions or four trigonometric functions, the method that should be employed depends on which is faster to compute.

Fig. 1 Lambert problem geometry



3 Universal Lambert solver

In this section, Lambert's problem is properly introduced and the proposed solution method is presented. To begin, examine Fig. 1, as it shows the standard geometry considered when solving Lambert's problem. The goal is to determine the initial and final velocities, $\mathbf{v}_0$ and $\mathbf{v}_f$, from the known initial and final positions, $\mathbf{r}_0$ and $\mathbf{r}_f$, and a known time of flight/transfer time, $\Delta t = t_f - t_0$. The transfer angle (θ) must also be considered, as two transfer directions exist, one in the posigrade direction and one in the retrograde. The transfer angle may be determined by

$$\theta = 2\pi\eta + (-1)^\eta \arccos \frac{\mathbf{r}_f \cdot \mathbf{r}_0}{r_f r_0} \quad (26)$$

where η is used to determine the direction of transfer. $\eta = 0$ for the short-way ($\theta < \pi$) and $\eta = 1$ for the long way $\theta > \pi$. The opposed direction transfer direction is denoted as $\tilde{\theta} = 2\pi - \theta$ in this work, as it is of use when discussing multi-revolution transfers. If the solutions for both directions are desired for a given geometry and transfer time, each must be treated separately, as the choice of transfer angle governs not only the direction of motion but also the formulation of the initial guess and the computation of the geometric constant used in the transfer time equation. As a result, the solver performs nearly all steps independently for each case, yielding minimal opportunity for reuse or computational savings.

Once a direction of transfer is assumed, an initial guess is made on the universal variable, and the transfer time is solved for by utilizing the two given positions. The error between the desired and calculated transfer time is then determined, and an iterative root solver is generally employed to determine an update to the universal variable until a convergence criterion is reached. As has been shown in many solvers before, the transfer time equation can be expressed as a function of a single unknown (Lancaster and Blanchard 1969; Bate et al. 2020; Arora and Russell 2013; Izzo 2015). In this work, the single unknown in the transfer time equation is defined to be:

$$x = \sqrt{\alpha}\chi = \begin{cases} \Delta E & \text{for elliptic cases} \\ j\Delta H & \text{for hyperbolic cases} \end{cases} \quad (27)$$

where ΔE is the difference in eccentric anomaly and ΔH is the difference in hyperbolic anomaly. The universal variable is similar to the work of Bate et al., as they defined their universal variable $z = x^2$. The reason for selecting x instead of z is that the universal functions are better represented as functions of x , reducing the complexity of the transfer time partial derivatives.

3.1 Transfer time equation

Using Eqs. 6 and 20, it is known that Kepler's equation can be assembled through universal functions, yielding:

$$\sqrt{\mu}\Delta t = U_1 r_0 + U_2 \sigma_0 + U_3 \quad (28)$$

Currently, σ_0 is not known in the context of this problem, as it depends on the velocity at the initial time. To remove this dependency, the geometry of the problem is utilized. In Battin (1999), eqn. 4.98, it is shown that the cosine of half the transfer angle can be expressed as:

$$\cos \frac{1}{2}\theta = \frac{1}{\sqrt{r_f r_0}} (r_0 U_0^* + \sigma_0 U_1^*) \quad (29)$$

Rearranging Eq. 29 and solving for σ_0 yields:

$$\sigma_0 = \frac{m - r_0 U_0^*}{U_1^*} \quad (30)$$

where

$$m = \sqrt{r_f r_0} \cos \frac{1}{2}\theta \quad (31)$$

Note that the determination of m is the only time that a trigonometric function *must* be employed when calculating the transfer time. Since m is a pure function of the problem geometry, it can be determined *outside of the iteration process*, reducing the total required number of trigonometric function evaluations to one. With σ_0 defined as a function of the geometry and universal functions, the transfer time equation can be rewritten as:

$$\sqrt{\mu}\Delta t = U_1 r_0 + \frac{U_2}{U_1^*} (m - r_0 U_0^*) + U_3 \quad (32)$$

To simplify the transfer time equation further, the universal function identities of Eqs. 24 and 25 are employed, reducing the transfer time equation to its simplest form.

$$T(x) = \sqrt{\mu}\Delta t = 2U_1^* m + U_3 \quad (33)$$

T is the scaled transfer time and is used for concise notation. Equation 33 is the final form of the transfer time equation employed in this work, where the equation captures single-revolution hyperbolic/elliptic and multi-revolution elliptic orbits. While not needed in this work, an equivalent form of the transfer time equation for multi-revolution solutions is usually given by:

$$T(x) = 2U_1^* m + U_3 + \frac{2\pi N}{\alpha^{3/2}} \quad (34)$$

where N is the number of completed revolutions. Note that the additional term between Eqs. 33 and 34 is referred to by the authors as the multi-revolution term. Later, it is shown how the value of the universal variable x can yield multi-revolution solutions from Eq. 33 without the need for Eq. 34.

To a keen-eyed reader, it is quickly apparent that transfer time is still a function of two unknowns: α and χ . While a method could be derived to solve for these two variables simultaneously, instead, expressions for α and χ as a function of the universal variable x

are desired. To determine an equation for α , begin with an expression that relates the initial position to the final (eqn. 4.82 of Battin 1999),

$$r_f = r_0 U_0 + \sigma_0 U_1 + U_2 \quad (35)$$

Using the information in Eqs. 30 and 18 then yields:

$$r_f = r_0 + \frac{U_0}{U_1^*} (m - r_0 U_0^*) + U_2 (1 - \alpha r_0) \quad (36)$$

Applying the universal function identity in Eq. 25 and simplifying the resulting expression gives:

$$r_f + r_0 = 2mU_0^* + U_2 \quad (37)$$

Using the definition of U_2 in Eq. 18, an expression for α as a function x is determined to be:

$$\alpha = \frac{1 - U_0}{r_f + r_0 - 2mU_0^*} \quad (38)$$

While at first glance the expression appears to still be a function of both χ and α , both U_0 and U_0^* can both be determined from the universal variable x .

$$U_0(x) = U_0(\chi; \alpha) = \frac{1}{2} \left(e^{j\sqrt{\alpha}\chi} + e^{-j\sqrt{\alpha}\chi} \right) = \frac{1}{2} \left(e^{jx} + e^{-jx} \right) \quad (39)$$

$$U_0^*(x) = U_0\left(\frac{1}{2}\chi; \alpha\right) = \frac{1}{2} \left(e^{\frac{1}{2}j\sqrt{\alpha}\chi} + e^{-\frac{1}{2}j\sqrt{\alpha}\chi} \right) = \frac{1}{2} \left(e^{\frac{1}{2}jx} + e^{-\frac{1}{2}jx} \right) \quad (40)$$

Thus, α is obtainable from a value of x and χ can be recovered simply by:

$$\chi = \frac{x}{\sqrt{\alpha}} \quad (41)$$

With both α and χ values attainable from x , the universal functions can all be evaluated and then used to calculate the transfer time.

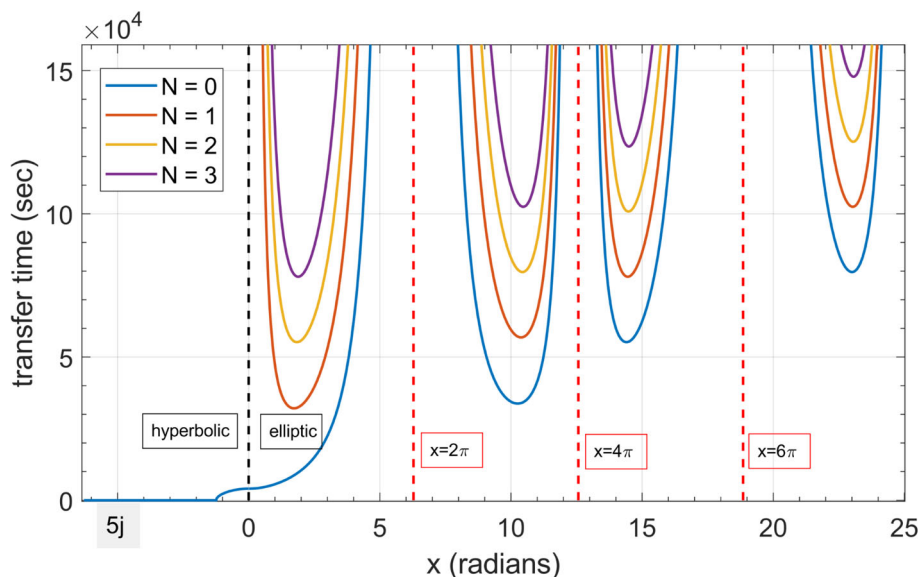
3.1.1 Removal of multi-revolution term

Plots of the transfer time as a function of x are now examined. The values for position, transfer angle, and gravitational parameter/equatorial radius of the Earth are given in Table 1. The transfer time is plotted for θ and $\tilde{\theta}$ in Figs. 2 and 3, respectively. Note that Eq. 34 was used in these plot generations. Firstly, the values on the x-axis to the left of zero are imaginary values, not negative, as hyperbolic solutions for x are imaginary. Second, it is seen that an infinite number of solutions exist as x goes to positive infinity with vertical asymptotes at increments of 2π . While one might infer that solutions separated by each asymptote refer to an increase in the number of revolutions, that would be incorrect. The profiles do correspond to an increase in the number of revolutions but for alternating transfer directions. This is evident when comparing Figs. 2 and 3, for the $N = 1$ solution for θ and $0 < x < 2\pi$ is equivalent to the $N = 0$ solution for $\tilde{\theta}$ and $2\pi < x < 4\pi$, the $N = 2$ solution for $\tilde{\theta}$ and $2\pi < x < 4\pi$ is equivalent to the $N = 1$ solution for θ and $4\pi < x < 6\pi$, and so on. To capture this behavior in the transfer time calculation, the constant m is determined by:

$$m = (-1)^N \sqrt{r_0 r_f} \cos \frac{1}{2}\theta \quad (42)$$

Table 1 Parameter values for Figs. 2 and 3

r_0	r_f	θ	$\tilde{\theta}$	r_e	μ
$2r_e$	$5r_e$	45°	315°	6378 km	$3.986\text{E}5 \text{ km}^3/\text{s}^2$

**Fig. 2** Plot of transfer time versus x for θ

since:

$$\cos \frac{1}{2} \tilde{\theta} = \cos \frac{1}{2} (2\pi - \theta) = \cos \left(\pi - \frac{1}{2} \theta \right) = -\cos \frac{1}{2} \theta \quad (43)$$

Thus, if m is calculated by Eq. 42 and the universal variable selected within the appropriate bounds, multi-revolution transfers can be solved using Eq. 33. This is an important reduction, as removing the additional term in Eq. 34 reduces both the analytical and computational complexity of the transfer time function derivatives. The reduction in derivative complexity makes the use of higher-order correction methods (such as third-order Householder) more attractive. The removal of the term also improves numerical stability of the transfer time function, as in Eq. 34, the multi-revolution term $\left(\frac{2\pi N}{\alpha^{3/2}} \right)$ grows significant larger in magnitude than $2U_1^* m$ and U_3 at large numbers of revolutions.

3.2 Initial Guess schemes

When developing the initial guess schemes for each Lambert solution branch (hyperbolic, single-revolution elliptic, and multi-revolution), the primary goal is to provide an accurate yet computationally inexpensive approximation of the true transfer time profile. This is accomplished by constructing simplified “curve fits” that preserve essential features of the exact solution—such as known anchor points (e.g., the parabolic transfer time, the minimum transfer time, or the time associated with the transfer angle) and asymptotic behaviors (e.g., vertical asymptotes at integer multiples of 2π for multi-revolution transfers). By ensuring

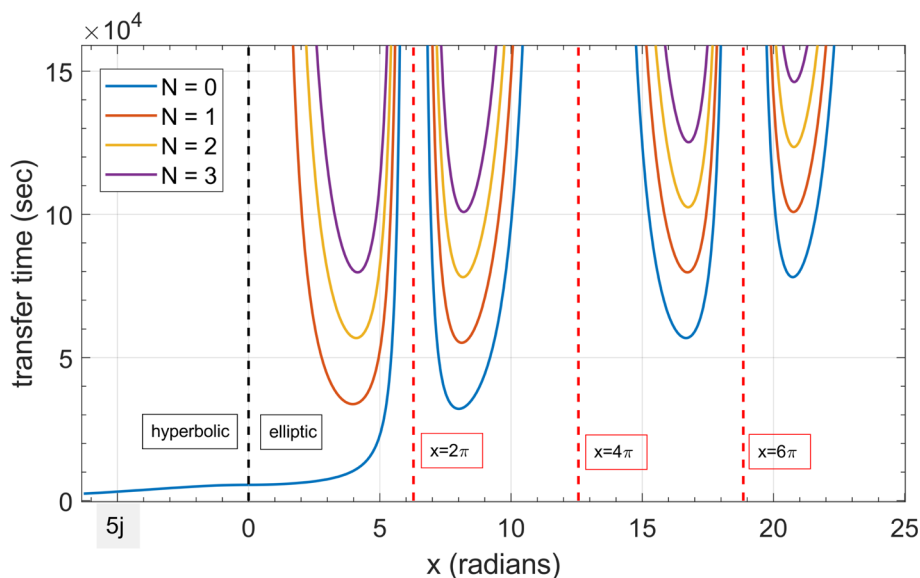


Fig. 3 Plot of transfer time versus x for $\tilde{\theta}$

that each approximating function smoothly transitions between these key values and correctly reproduces the overall shape of the transfer time curves observed in Figs. 2 and 3, each scheme remains sufficiently close to the actual solution to serve as a robust initial guess. This general procedure—anchoring simple functions to known points, enforcing asymptotes, and matching the characteristic behavior of each solution branch—guides the specific functional forms described in the subsequent sections. These carefully chosen approximations significantly reduce the number of iterations required by the Lambert solver, thus improving both efficiency and reliability across a wide range of orbital geometries and revolution counts.

Because the coefficients of the approximating functions are calculated such that the profiles pass through solutions corresponding to the transfer angle, the following relations are defined:

$$T_{j\theta} = T(x_{j\theta}) \quad ; \quad x_{j\theta} = j\theta \quad (44)$$

$$T_{\theta} = T(x_{\theta}) \quad ; \quad x_{\theta} = 2\pi N + \theta \quad (45)$$

where these definitions provide a more concise notation. Note that forcing the approximating functions to pass through values corresponding to the transfer angle yields excellent initial guesses for low eccentricity orbits (i.e., $\Delta E \approx \theta + 2\pi N$).

3.2.1 Single-revolution

For the Lambert solver, a simple-to-implement initial guess scheme is employed for hyperbolic trajectories and single-revolution elliptic orbits ($N = 0$). While one may desire one solution over the other, the given transfer time determines the conic of the solution. As can be seen in Figs. 2 and 3, as x approaches zero from the right, the solution transitions from elliptic to hyperbolic. Thus, if the given transfer time $\Delta \tilde{t}$ is greater than parabolic transfer time Δt ($x = 0$), then the solution is elliptic. If the given transfer time is less than the parabolic transfer time, the solution is hyperbolic. Note, however, that a singularity exists at $x = 0$ due

to α becoming zero when the solution is parabolic. This singularity is not unique to this formulation, as most Lambert solvers contain a singularity at the solution corresponding to the parabolic transfer time. However, determining the parabolic transfer time is computationally cheap, as the analytical solution can be easily derived. To determine the parabolic transfer time, the universal functions are evaluated using their power series representation, Eq. 12, with $\alpha = 0$.

$$U_0(\chi; 0) = 1; \quad \& \quad U_0\left(\frac{\chi}{2}; 0\right) = U_0^*(\chi; 0) = 1 \quad (46)$$

$$U_1(\chi; 0) = \chi; \quad \& \quad U_1\left(\frac{\chi}{2}; 0\right) = U_1^*(\chi; 0) = \frac{\chi}{2} \quad (47)$$

$$U_2(\chi; 0) = \frac{\chi^2}{2} \quad \& \quad U_3(\chi; 0) = \frac{\chi^3}{6} \quad (48)$$

χ is also evaluated for the case when $\alpha = 0$ yielding:

$$\chi(\alpha = 0) = \chi_0 = \sqrt{2(r_f + r_0 - 2m)} \quad (49)$$

The evaluation of χ when $\alpha = 0$ is shown in Appendix B. Thus, the scaled parabolic transfer time ($T_0 = T(0)$) can be calculated by Eq. 50. If the given transfer time equals the parabolic transfer time, then the solution is parabolic and no iterations are necessary.

$$T_0 = m\chi_0 + \frac{\chi_0^3}{6} \quad (50)$$

When solving for a hyperbolic solution, a different initial guess scheme is employed depending on whether the transfer angle is less than or greater than 180° . In both cases, an interpolation scheme is used, with the approximating function different between the two. For the $\theta \geq \pi$ case (imaginary region of Fig. 3), the function used to approximate the transfer time is defined to be:

$$T(x) \approx T_0 e^{\beta_1(-jx)^2} \quad (51)$$

where

$$\beta_1 = \frac{1}{\theta^2} \log \frac{T_{j\theta}}{T_0} \quad (52)$$

Note that in this work “log” is the natural logarithmic function. The approximating function was constructed to enforce the following assumptions: T exponentially decays as $x \rightarrow j\infty$, the function equals T_0 when x equals zero, and the profile passes through $T_{j\theta}$. To determine the initial guess, Eq. 51 is set equal to the given scaled transfer time ($\tilde{T} = \sqrt{\mu} \Delta \tilde{t}$) and x is solved for, yielding:

$$x_0 = j\theta \sqrt{\frac{\log \frac{\tilde{T}}{T_0}}{\log \frac{T_{j\theta}}{T_0}}} \quad (53)$$

For the cases when $\theta < \pi$ (imaginary region of Fig. 2), the function used to approximate the transfer time is defined to be:

$$T(x) \approx \frac{T_0}{\beta_3^{5/8}} (\beta_3 - (-jx)^{\beta_2})^{5/8} \quad (54)$$

where

$$\beta_2 = \frac{2}{5} \log \frac{-T_0^{8/5}}{T(x_\tau e^{-2.5})^{8/5} - T_0^{8/5}} ; \quad \beta_3 = (-jx_\tau)^{\beta_2} \quad (55)$$

x_τ is the value at which the transfer time function becomes zero for the $\theta < \pi$ hyperbolic cases and is calculated by:

$$x_\tau = j \log \left(\tau + \sqrt{\tau^2 - 1} \right) ; \quad \tau = \frac{r_0 + r_f}{2m^2} - 1 \quad (56)$$

The rationale behind the construction of this approximating function is that the transfer time profile for $\theta < \pi$ hyperbolic cases resembles a square root-like curve. An exponent of $5/8$ is used instead of $1/2$ because it was empirically found to better match the shape of the transfer time curve during testing. The function is designed to satisfy key conditions: it evaluates to zero at x_τ , equals the parabolic transfer time at $x = 0$, and passes through the reference point $T(x_\tau e^{-2.5})$. To determine the initial guess, Eq. 54 is set equal to the given scaled transfer time and x is solved for, yielding:

$$x_0 = x_\tau \left[1 - \frac{\tilde{T}}{T_0} \right]^{1/\beta_2} \quad (57)$$

As will be shown in the numerical results section, these two initial guess schemes are accurate for eccentricities up to 100.

For the single-revolution elliptic cases ($N = 0$ profile for $0 \leq x \leq 2\pi$ in Figs. 2 and 3), one approximating function is used for the transfer time regardless of the transfer angle. The approximating function is defined to be:

$$T(x) \approx (T_\theta - T_0) \left(\frac{2\pi - x_\theta}{x_\theta} \right)^2 \frac{x^2}{(2\pi - x)^2} + T_0 \quad (58)$$

This approximation was employed as it preserves the vertical asymptote at $x = 2\pi$ and passes through T_0 and T_θ . To determine the initial guess for the single-revolution elliptic cases, Eq. 58 is set equal to the given scaled transfer time and x is solved for, yielding:

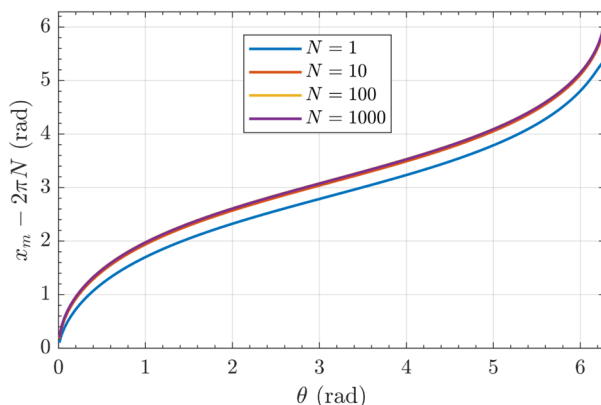
$$x_0 = \frac{2\pi\beta_4}{1 + \beta_4} ; \quad \beta_4 = \frac{x_\theta}{2\pi - x_\theta} \sqrt{\frac{\tilde{T} - T_0}{T_\theta - T_0}} \quad (59)$$

With the single-revolution cases addressed, the attention of the work is now shifted to multi-revolution initial guesses.

3.2.2 Multi-revolution

For multi-revolution transfers, two solutions exist for each transfer direction. The two-solution behavior for each transfer direction is apparent when looking at Figs. 2 and 3, as each multi-revolution solution exhibits two values for each transfer time, true for all transfer times greater than the minimum transfer time (T_m). To solve for the two solutions, one must ensure that the given transfer time is greater than the minimum, as trying to solve for a root with a transfer time less than the minimum yields incorrect solutions. While others have approximated T_m in their Lambert solvers for speed (Izzo 2015), it is best practice to determine the minimum transfer time exactly, generally completed through numerical methods.

Fig. 4 Plot of the average value of $x_m - 2\pi N$ versus transfer angle



In this work, Halley's method is employed to determine both T_m and its associated x value (x_m). The initial guess for this first root solve is:

$$x_{m0} = \begin{cases} 2 \left(\theta + \left(\frac{1}{6} \right)^{2.5\sqrt{N}} \right)^{1/2.5} - \left(\frac{1}{3} \right)^{\sqrt{N}} + 2\pi N & \text{for } \theta < \pi \\ -2 \left(\theta + 2\pi + \left(\frac{1}{20} \right)^{\sqrt{N}} \right)^{1/2.5} - \left(\frac{1}{4} \right)^{\sqrt{N}} + 2\pi(N+1) & \text{for } \theta \geq \pi \end{cases} \quad (60)$$

The initial guess scheme for x_m was selected as by investigating $x_m - 2\pi N$ as a function of the transfer angle, averaged over a wide range of transfer geometries, Fig. 4. From the figure, it was seen that the value of $x_m - 2\pi N$ was relatively well behaved and collapsed to a single profile as N increased. Thus, reasonable approximations of x_m can be generated by determining functions of θ that yield the resulting profiles, Eq. 60. Once the minimum transfer time is determined by performing Halley iterations, one proceeds knowing whether two solutions exist. Assuming that the given transfer time is greater than T_m , initial guesses are made on each side of the x_m .

The authors have found the following multi-revolution initial guess scheme to be quite robust. Utilizing x_m , initial guesses can be generated on either side of its value, one short period and one long period solution. The approximating function for multi-revolution solutions is formulated under the following criteria: the multi-revolution profile is bounded between two asymptotes at $2\pi N$ and $2\pi(N+1)$, $T(x)$ must equal T_m at x_m , $T(x)$ should pass through T_θ , and the derivative of T at x_m must equal zero. A reasonable approximation for T can be defined as:

$$T(x) \approx \beta_5 \frac{(x_m - x)^2}{(2\pi N - x)^2(2\pi(N+1) - x)^2} + T_m \quad (61)$$

where

$$\beta_5 = \frac{|T_\theta - T_m|(2\pi N - x_\theta)^2(2\pi(N+1) - x_\theta)^2}{(x_m - x_\theta)^2} \quad (62)$$

To determine the two multi-revolution initial guesses, Eq. 61 is set equal to the given scaled transfer time and x is solved for using the well-known quadratic equation, yielding:

$$x_0 = \frac{-c_2 + \sqrt{c_2^2 - 4c_1c_3}}{2c_1} \quad (63)$$

where

$$c_1 = \gamma \quad ; \quad c_2 = 1 - 2\pi(2N + 1)\gamma \quad ; \quad c_3 = 4\pi^2 N(N + 1)\gamma - x_m \quad (64)$$

$$\gamma = \begin{cases} -\sqrt{\frac{\tilde{T} - T_m}{\beta_5}} & \text{for the } x < x_m \text{ solution} \\ +\sqrt{\frac{\tilde{T} - T_m}{\beta_5}} & \text{for the } x > x_m \text{ solution} \end{cases} \quad (65)$$

With the initial guesses for each method presented, the root solvers employed in this work are now discussed.

3.3 Root solvers

Two root solvers are employed in this work: Halley's method to solve for the minimum transfer time of multi-revolution solutions and a third-order Householder's method to solve for solutions corresponding to the desired transfer time. To determine the minimum transfer time, the derivative of the transfer time equation is set equal to zero.

$$g(x) = 0 = \frac{\partial T}{\partial x} \quad (66)$$

To find the minimum transfer time, the value of x that solves Eq. 66 is needed. An initial guess is then made on x using the method discussed in the previous section, and is then updated using Halley's iterative method:

$$x_{n+1} = x_n - \frac{2g(x_n)g'(x_n)}{2g'(x_n)^2 - g(x_n)g''(x_n)} \quad (67)$$

where

$$g'(x_n) = \frac{\partial^2 T}{\partial x^2} \big|_{x_n} \quad \text{and} \quad g''(x_n) = \frac{\partial^3 T}{\partial x^3} \big|_{x_n} \quad (68)$$

Note that in this work, the prime notation ($[...]'$) indicates a derivative with respect to the universal variable x . The full expressions for the partial derivatives of the transfer time equation are given in Appendix A of this work. The partial derivatives are evaluated for the current value of x and then are used to update x until a convergence criterion is met. The relative difference between x_{n+1} and x_n becoming less than some prescribed small value (ϵ) was used:

$$\epsilon > \left| \frac{x_{n+1} - x_n}{x_{n+1} - 2\pi(N - 1)} \right| \quad (69)$$

Note the use of $2\pi(N - 1)$ in the denominator, as this prevents the convergence criterion from becoming lax when N becomes large.

To update the initial guess/current value of x when solving for the desired Lambert solution, a third-order Householder's method root solver is employed. First, a function is defined as the difference between the calculated scaled transfer time and the given scaled transfer time.

$$h(x) = T - \tilde{T} \quad (70)$$

As is apparent from the formulation, a solution to Lambert's problem appears once the value of $h(x)$ becomes zero. To find a root to the equation, an initial guess on x is made and then updated using the following scheme:

$$x_{n+1} = x_n - \frac{6h(x_n)h'(x_n)^2 - 3h''(x_n)^2h(x_n)}{6h'(x_n)^3 - 6h(x_n)h'(x_n)h''(x_n) + h(x_n)^2h'''(x_n)} \quad (71)$$

where

$$h'(x_n) = \frac{\partial T}{\partial x}|_{x_n} \quad \text{and} \quad h''(x_n) = \frac{\partial^2 T}{\partial x^2}|_{x_n} \quad \text{and} \quad h'''(x_n) = \frac{\partial^3 T}{\partial x^3}|_{x_n} \quad (72)$$

The updates continue until a convergence criterion is met. When doing standard implementations, the authors recommend the same convergence criterion that was given in Eq. 69. Note that the authors have found that using a value of 10^{-10} for ϵ provides a velocity accuracy of at least 12 digits for most cases. Once the convergence criterion is met, the desired velocities can be calculated.

3.4 Reconstructing velocity vectors

To reconstruct the velocity vectors after converging to a solution, the Stark (1961) transformation is utilized. The Stark transformation is a non-orthogonal coordinate transformation that expresses two velocity vectors in terms of vector components with the same magnitude. These components are determined by:

$$v_c = \frac{c\sqrt{\mu p}}{r_1 r_2 \sin \theta} \quad ; \quad v_\rho = \sqrt{\frac{\mu}{p}} \frac{1 - \cos \theta}{\sin \theta} \quad (73)$$

where

$$p = \frac{r_0 r_f (1 - \cos \theta)}{r_0 + r_f - 2mU_0^*} \quad ; \quad c = ||\mathbf{r}_f - \mathbf{r}_0|| \quad (74)$$

Once the components are calculated, the velocity vectors are quickly reconstructed by:

$$\mathbf{v}_1 = v_c \hat{\mathbf{i}}_c + v_\rho \hat{\mathbf{i}}_{r_0} \quad ; \quad \mathbf{v}_2 = v_c \hat{\mathbf{i}}_c - v_\rho \hat{\mathbf{i}}_{r_f} \quad (75)$$

with the unit directions defined as:

$$\hat{\mathbf{i}}_{r_0} = \frac{\mathbf{r}_0}{r_0} \quad ; \quad \hat{\mathbf{i}}_{r_f} = \frac{\mathbf{r}_f}{r_f} \quad ; \quad \hat{\mathbf{i}}_c = \frac{\mathbf{r}_f - \mathbf{r}_0}{c} \quad (76)$$

Note that when $\theta = \pi$, the components v_c and v_ρ become ∞ due to the sine of the transfer angle being in the denominator. This singularity is common to all Lambert solvers, as reconstructing velocity vectors when $\theta = \pi$ is impossible without *a priori* knowledge of the orbit plane. If the orbit plane is known, the velocity can be reconstructed. However, utilizing the half-revolution solutions in a Lambert solver is problematic as they are only valid for the exact half-rev case, the orbit plane must be known, and the transition between cases is not generally smooth. Thus, the half-revolution cases are ignored. Users needing the exact half-revolution are recommended to refer to pg. 241 of Battin (1999). With the reconstruction of the velocity vectors shown, the authors now detail a few items about the Lambert solver implementation.

4 Lambert solver implementation

Before moving to the numerical results, the authors wish to share a few notes on the solver implementation and then provide an algorithm for the reader's benefit. The notes on implementation address some edge cases that need to be mitigated, as well as how solver speed can be improved. The algorithms presented closely match the implementation of the source code provided.

4.1 Solving for x_m

Since the determination of x_m is most often used for multi-revolution initial guesses, an extremely precise value is not needed for most cases. A precise value of x_m is needed when the minimum transfer time is close to the given transfer time, so the authors chose to use a variable value of ϵ when solving for x_m . This allows the solver to be precise when needed but relaxed when not. To implement this, a check is performed during each iteration when solving for x_m , where it is determined if the given transfer time is either less than or close to the current iteration value of T_m . If so, the tolerance is increased to ensure an accurate value of T_m . The authors found that the following values work well:

$$\epsilon = \begin{cases} 10^{-2} & \text{when } 1 - \frac{T_m}{T} > 10^{-4} \\ 10^{-8} & \text{when } 1 - \frac{T_m}{T} < 10^{-4} \end{cases} \quad (77)$$

For the most robust performance, the ϵ value should be small for all solves of x_m , not just when T_m is near the given transfer time. However, in testing, none of the short/long period initial guesses for multi-revolution solutions ended up converging to the same solution.

4.2 Reduced order update

In some rare instances, the Householder and Halley updates can cause the updated value of x or x_m to exceed the known bounds of the solve space. For example, single-revolution updates causing x to become less than zero or greater than 2π . This most often occurs when the transfer angle is close to 0 or 2π or when transferring from one hyperbolic asymptote to another. These are known stress cases of Lambert's problem, as the true solutions are close to the solution bounds. When a crossing of the bounds occurs, the authors have seen that simply reverting to a lower-order Householder update resolves the issue. First the second-order (Halley's method) is used:

$$x_{n+1} = x_n - \frac{2h(x_n)h'(x_n)}{2h'(x_n)^2 - h(x_n)h''(x_n)} \quad (78)$$

and then, if needed, the first-order (Newton's method):

$$x_{n+1} = x_n - \frac{h(x_n)}{h'(x_n)} \quad (79)$$

A quick check is performed in the algorithm such that if the update exceeds the solution bounds, updates are applied using a lower-order Householder update.

4.3 Complex arithmetic in the hyperbolic regime

In the elliptic regime, nearly all quantities remain real-valued, aside from a brief use of complex exponentials during universal function evaluation. In the hyperbolic regime, the universal variable x is purely imaginary, which causes certain intermediate quantities—such as the first and third derivatives of the transfer time function and of α with respect to x —to be complex. The use of complex arithmetic is localized to the initial guess generation and the iteration phase of the solver for hyperbolic solutions, while all physical outputs, including reconstructed velocity vectors, remain in the real domain.

4.4 Velocity derivatives with respect to inputs

Although this work does not derive them explicitly, the partial derivatives of the initial and final velocities with respect to the input parameters— $\mathbf{r}_0$, $\mathbf{r}_f$, and Δt —are expected to be analytically tractable within the proposed formulation. These derivatives would require careful application of the chain rule, as the velocities depend implicitly on the universal variable and transfer time function. The closed-form nature of the solver makes such differentiation feasible, though nontrivial, particularly in the presence of higher-order terms. These expressions could be valuable in future work involving trajectory optimization or state estimation frameworks that require sensitivity analysis.

4.5 Algorithm

To the benefit of the reader, an algorithm that contains the essential information to implement this Lambert solver is presented in Algorithm 1.

Algorithm 1 Lambert solver pseudo code

Require: Given $\mathbf{r}_0$, $\mathbf{r}_f$, $\Delta \tilde{t}$, η , N , μ
Ensure: Calculate $\mathbf{v}_0$, $\mathbf{v}_f$

- 1: Calculate θ by Eq. 26.
- 2: $\tilde{T} = \sqrt{\mu} \Delta \tilde{t}$
- 3: **if** $N = 0$ **then**
- 4: Determine T_0 by Eq. 50.
- 5: **if** $T_0 = \tilde{T}$ **then**
- 6: $x = 0$.
- 7: Use x to calculate $\mathbf{v}_0$ and $\mathbf{v}_f$ [Eqs. 73–76] and exit algorithm early.
- 8: **else if** $T_0 < \tilde{T}$ **then**
- 9: Determine x_0 from Eq. 59.
- 10: **else if** $T_0 > \tilde{T}$ **then**
- 11: **if** $\theta \geq \pi$ **then**
- 12: Determine x_0 from Eq. 53.
- 13: **else**
- 14: Determine x_0 from Eq. 57.
- 15: **end if**
- 16: **end if**
- 17: **else**
- 18: Determine x_{m0} by Eq. 60.
- 19: Starting with x_{m0} , iteratively update x_m by Eq. 67 until converging.
- 20: Calculate T_m from x_m .
- 21: **if** $\tilde{T} < T_m$ **then**
- 22: The given transfer time not feasible; exit algorithm and throw error.
- 23: **else if** $\tilde{T} = T_m$ **then**
- 24: $x = x_m$.
- 25: Use x to calculate $\mathbf{v}_0$ and $\mathbf{v}_f$ [Eqs. 73–76] and exit algorithm early.
- 26: **else**
- 27: Determine the two values for x_0 by Eqs. 63–65.
- 28: **end if**
- 29: **end if**
- 30: **for** $i = 1$ to $\text{size}(x_0)$ **do**
- 31: Starting with $x_0(i)$, iteratively determine $x(i)$ by Eq. 71 until converging.
- 32: Use $x(i)$ to calculate $\mathbf{v}_0(i)$ and $\mathbf{v}_f(i)$ [Eqs. 73–76].
- 33: **end for**

5 Results

To demonstrate the effectiveness of the proposed solver, it was compared against the method of Gooding (1990)—widely considered the benchmark—across a broad range of representative cases. Both Gooding's and the method presented were written and implemented in MATLAB R2024b, with Gooding's algorithm being converted from the original FORTRAN code. Modifications were made to Gooding's method to reconstruct the full 3D velocity vector and not just the radial and transverse components. For both methods, all test cases were solved using loops and not vectorization to provide the best one-to-one comparison. Computations were performed using a computer with an Intel Core i7-14700F CPU, an NVIDIA GeForce RTX 4060 GPU, and 32.0 GB of RAM.

Transfer geometries for this analysis were generated over a large number of test cases. These cases were generated by first selecting a fixed semimajor axis of 10 times the Earth's radius (negative 10 times for hyperbolic geometries). The semimajor axis was fixed as the results did not vary measurably when changing the semimajor axis, unsurprising as the semimajor axis only affects the problem scaling and not the transfer geometry. Transfer geometries were then formed for varying eccentricities across the full range of transfer angles and initial true anomalies. Eccentricities (e) were generated using a fixed step size of 0.01 for elliptic cases

$$e = [0.01 \ 0.02 \ 0.03 \ 0.04 \ \dots \ 0.99] \quad (80)$$

and 0.2 logarithmic step size up to 100 eccentricity for hyperbolic cases.

$$e = 10^{\wedge} [0.005 \ 0.02 \ 0.04 \ 0.06 \ \dots \ 2.0] \quad (81)$$

Because 10^0 gives the parabolic case, $10^{0.005}$ was used as the starting value. Although the proposed method supports parabolic transfers, these cases were excluded from the performance comparison with Gooding's method due to fundamental differences in their treatment. In the proposed algorithm, parabolic transfer times are computed analytically and, if matched, allow for immediate velocity reconstruction without any iteration. In contrast, Gooding's method does not include a distinct branch for parabolic cases and instead solves them iteratively. Including these cases would unfairly favor the proposed method in terms of speed, as it avoids iteration entirely.

To test over a large number of transfer locations, a parameter space was generated for the study, consisting of different combinations of initial true anomaly, f_0 , and transfer angle, θ . The initial true anomaly and transfer angle values were generated using a 0.5-degree step size, such that:

$$f_0 = [0.0 \ 0.5 \ 1.0 \ 1.5 \ \dots \ 360]^{\circ} \quad (82)$$

and

$$\theta = [0.5 \ 1.0 \ 1.5 \ 2.0 \ \dots \ 359.5]^{\circ} \quad (83)$$

Each combination of f_0 and θ was tested, leading to over 500,000 combinations for each eccentricity. Retrograde trajectories are implicitly included in the test suite through symmetry: for any retrograde transfer from f_0 to f_f , a corresponding prograde transfer with mirrored anomalies (e.g., $f'_0 = 2\pi - f_f$, $f'_f = 2\pi - f_0$) yields identical values for r_0 , r_f , θ , and Δt . The only difference lies in the direction of the reconstructed velocities, which does not affect the transfer geometry or timing.

Note that certain geometries generated by this method would lead to infeasible hyperbolic geometries, as hyperbolic transfer geometries are limited by the trajectory asymptotes, and were consequently thrown out. Because the initial true anomaly, final true anomaly ($f_f = f_0 + \theta$), and eccentricity were specified for the geometries, the true value of x was easily determined by converting each true anomaly to the eccentric/hyperbolic anomaly and taking the difference (which is equal to x). The relationship between true anomaly and eccentric anomaly is given as:

$$\tan \frac{1}{2}f = \sqrt{\frac{1+e}{1-e}} \tan \frac{1}{2}E \quad (84)$$

while the relationship between true and hyperbolic anomaly as:

$$\tan \frac{1}{2}f = \sqrt{\frac{e+1}{e-1}} \tanh \frac{1}{2}H \quad (85)$$

The true value of x in combination with the fixed semimajor axis ($1/\alpha$) was then used to determine the given transfer time $\tilde{T}$ and the true velocities ($\tilde{\mathbf{v}}_0$ and $\tilde{\mathbf{v}}_f$). The transfers were generated assuming Earth's gravitational parameter, μ in Table 1.

5.1 Reaching 10 digits of velocity accuracy

For the first analysis, each of the transfer geometries was tested for the proposed method and Gooding's, with the number of iterations required to reach 10 digits of velocity accuracy compared. To determine the iteration count, the following criteria were employed:

$$10^{-10} > \max \left[\frac{\|\tilde{\mathbf{v}}_0 - \mathbf{v}_0\|}{\|\tilde{\mathbf{v}}_0\|}, \frac{\|\tilde{\mathbf{v}}_f - \mathbf{v}_f\|}{\|\tilde{\mathbf{v}}_f\|} \right] \quad (86)$$

where the new criterion was assumed met once the relative velocity differences between the solver and true velocities were both below 10^{-10} . This ensured that convergence was not considered achieved until both velocities were accurate to at least 10 digits and required the velocity to be evaluated for each iteration. A vector-norm-based error criterion was used instead of an elementwise comparison to avoid numerical issues arising from division by near-zero velocity components, which can yield misleadingly large relative errors due to finite precision limits. While using velocity as a convergence criterion is not practical for real-world applications (as the truth is unknown), it is an attractive option when comparing methods as the scaling of the transfer time function and universal variable is different between most Lambert solvers. This also allowed the authors to determine if a solution simply failed to meet the desired velocity tolerances or if a solver converged to the wrong solution after the maximum number of iterations, 25. If the velocity error remained greater than five digits after the maximum number of iterations, it was assumed a solver converged to the wrong solution. If a failure occurred, that case was recorded as a "fail" and was not included when calculating the average or 99 percentile of iterations. Because multi-revolution transfer yields two solutions, only the solution that corresponded to the generated geometry was considered. The other solutions are still captured in the analysis as they simply correspond to a different transfer geometry.

As has been discussed, the minimum transfer time solution is determined before generating initial guesses for multi-revolution solutions. Because the outputs of this solve are generally only used for initial guesses, the minimum transfer time solution velocity is not reconstructed unless it is the solution itself. As such, comparing the number of digits required to reach a velocity accuracy for the minimum time solution does not provide much insight

Table 2 Number of single-revolution failed cases

	Hyperbolic (7,948,142 total)		Elliptic (51,179,040 total)	
	Gooding's	Proposed	Gooding's	Proposed
Failed to meet velocity accuracy	3002	171	280	0
Converged to wrong solution	0	0	24	0

into the performance of each solver. Instead, the number of iterations required to solve for the minimum transfer time was compared using each solver's individual convergence criteria. Gooding's method uses a fixed convergence criteria of the relative error in Gooding's universal variable (y) becoming less than 3×10^{-7} :

$$3 \times 10^{-7} > \left| \frac{y_{n+1} - y_n}{y_{n+1}} \right| \quad (87)$$

The proposed method uses a variable convergence criteria as described by Eqs. 69 and 77. By comparing the different convergence criteria, insights can be obtained on what the expected number of iterations will be during solves for the minimum time solution for each method.

5.1.1 Single-revolution results

For the single-revolution analysis, a total of 7,948,142 unique hyperbolic geometries and unique 51,179,040 elliptic geometries were tested, with hyperbolic being lower due to the removal of infeasible transfer geometries. Contour plots of iteration count for select eccentricities are shown in Figs. 5 and 6 for hyperbolic and elliptic cases, respectively. Note that the white spaces found in Fig. 5 indicate infeasible hyperbolic transfer geometries. In the contour plots, the true anomaly of the initial transfer location (f_0) is along the x-axis, the transfer angle (θ) is along the y-axis, and the number of iterations required to solve the geometry is shown by the varying colors. The left side of each figure gives the results of Gooding's method and the right the proposed method, where the eccentricity is varied along each vertical level.

With regard to Fig. 5, the first item pointed out is how Gooding's method had trouble solving when the transfer angle was small and the eccentricity was near 1. Gooding's method failed most often for these cases, unable to meet a velocity accuracy of 10 digits. In contrast, the proposed method was seen to have the most difficulty solving for transfers from one asymptote to another. When looking at the total number of failed cases, Table 2, Gooding's method was seen to fail about $15\times$ more often for hyperbolic cases. However, when looking at the average number of iterations and the 99 percentile of iterations, Figs. 7 and 8, Gooding's method was seen to perform well. The average number of iterations was less for Gooding's method above an eccentricity of ~ 1.25 , with the 99 percentile becoming two iterations. The proposed method solved, on average, in fewer iterations than Gooding's when near an eccentricity of one, but the average and 99 percentile iteration count increased with eccentricity, Fig. 7. Both methods stayed around an average of two iterations to solve, indicating a similar level of performance between the two methods. Gooding's method performed better at higher eccentricities while the proposed for eccentricities near 1, with the proposed method having fewer total failed cases.

With regard to Fig. 6, it was observed that Gooding's method again had trouble converging when the transfer angle was low and the eccentricity near a value of 1. Most of the time when

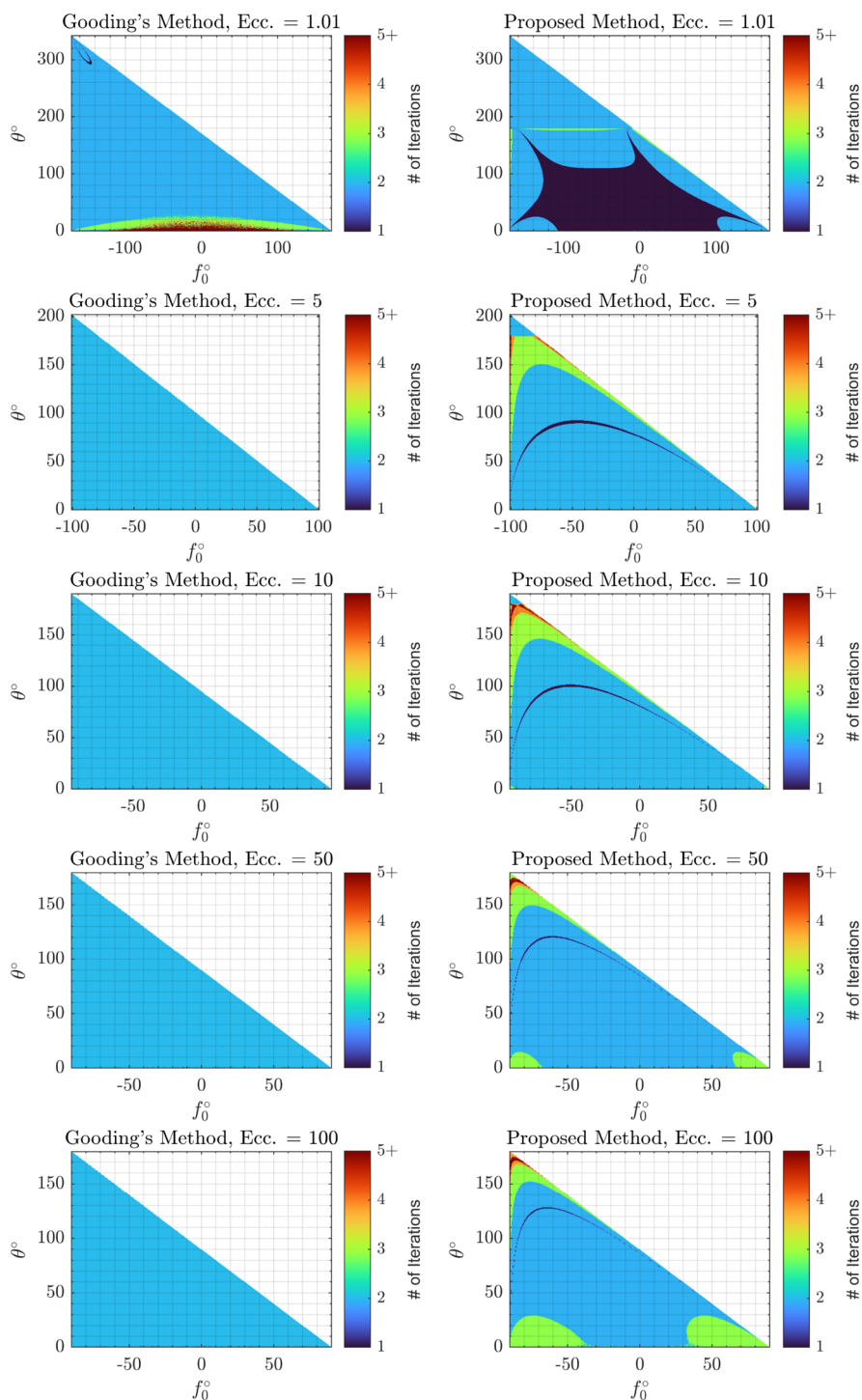


Fig. 5 Number of iterations for the hyperbolic cases to reach 10-digit velocity accuracy

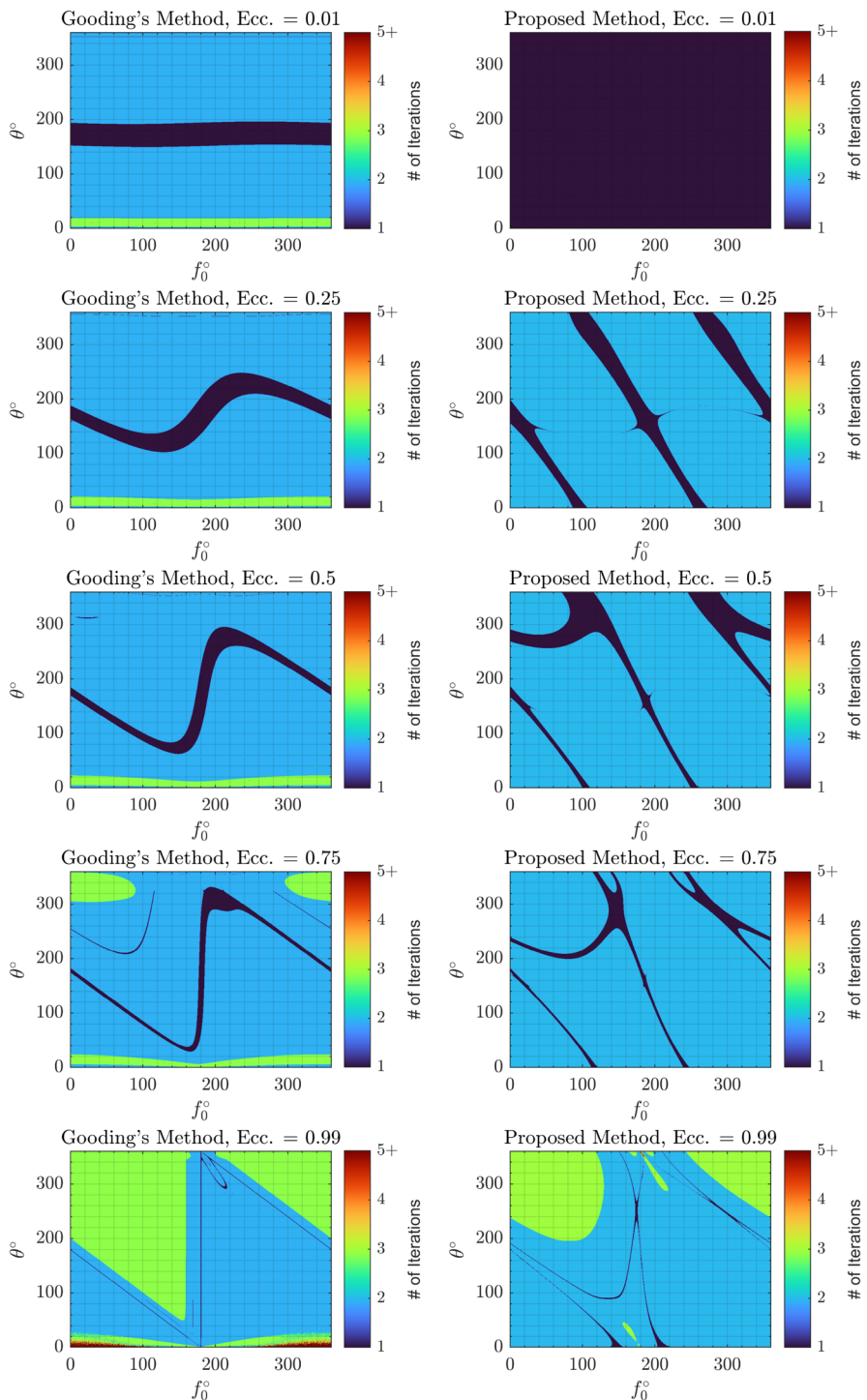


Fig. 6 Number of iterations for the $N = 0$ elliptic cases to reach 10-digit velocity accuracy

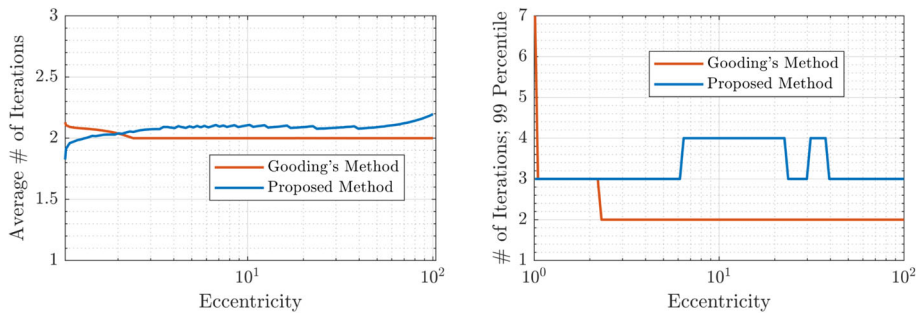


Fig. 7 Average and 99 percentile of solver iterations for hyperbolic cases

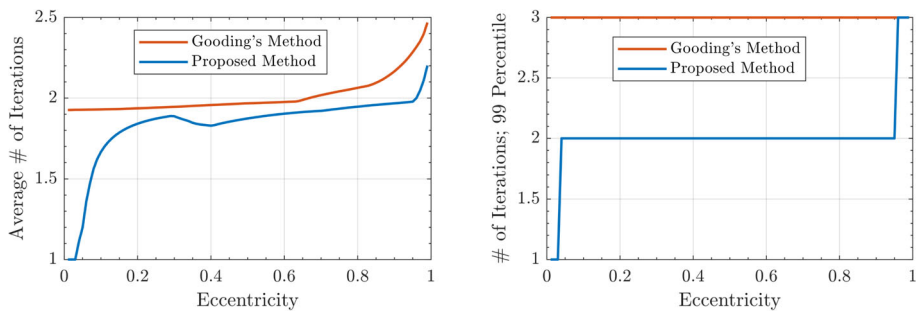


Fig. 8 Average and 99 percentile of solver iterations for single-revolution elliptic cases

Gooding's method failed, it was due to not meeting the desired velocity accuracy. However, in a few select cases, Gooding's method converged to the wrong solution. These cases were rare; only 24 out of ~ 50 million. In contrast, the proposed method did not fail a single solve, achieving the desired 10-digit velocity accuracy in two iterations or less for the vast majority of cases, Fig. 8. The proposed method was also seen to converge, on average, in fewer iterations for all eccentricities when compared to Gooding's method. The most notable takeaway from Figs. 6 and 8 is how the proposed method achieved 10 digits of velocity accuracy in, on average, 1 iteration for all geometries of 0.01, 0.02, and 0.03 eccentricity, an attractive advantage of this method as many real-world orbits are near-circular.

5.1.2 Multi-revolution results

For the multi-revolution analysis, 51,179,040 geometries were tested for each number of revolutions; a total of 307,074,240 cases ($N = 1, 10, 100, 1000, 10000, 100000$). Figures 9 and 10 give the number of iterations required to solve for the minimum transfer time and reach the velocity accuracy of the desired solution when $N = 1$, respectively. Figures 11 and 12 give the number of iterations required to solve for the minimum transfer time and reach the velocity accuracy of the desired solution when $N = 1000$, respectively. Again, the left side of each figure gives the results of Gooding's method and the right the proposed method, where the eccentricity is varied along the vertical levels.

With regard to Figs. 9 and 10, it was found that neither solver struggled with any geometries in particular when solving for x_m or x . Each solver had a few cases that failed to reach 10 digits of velocity accuracy, 49 for Gooding (Table 3), and 32 for the proposed (Table 4),

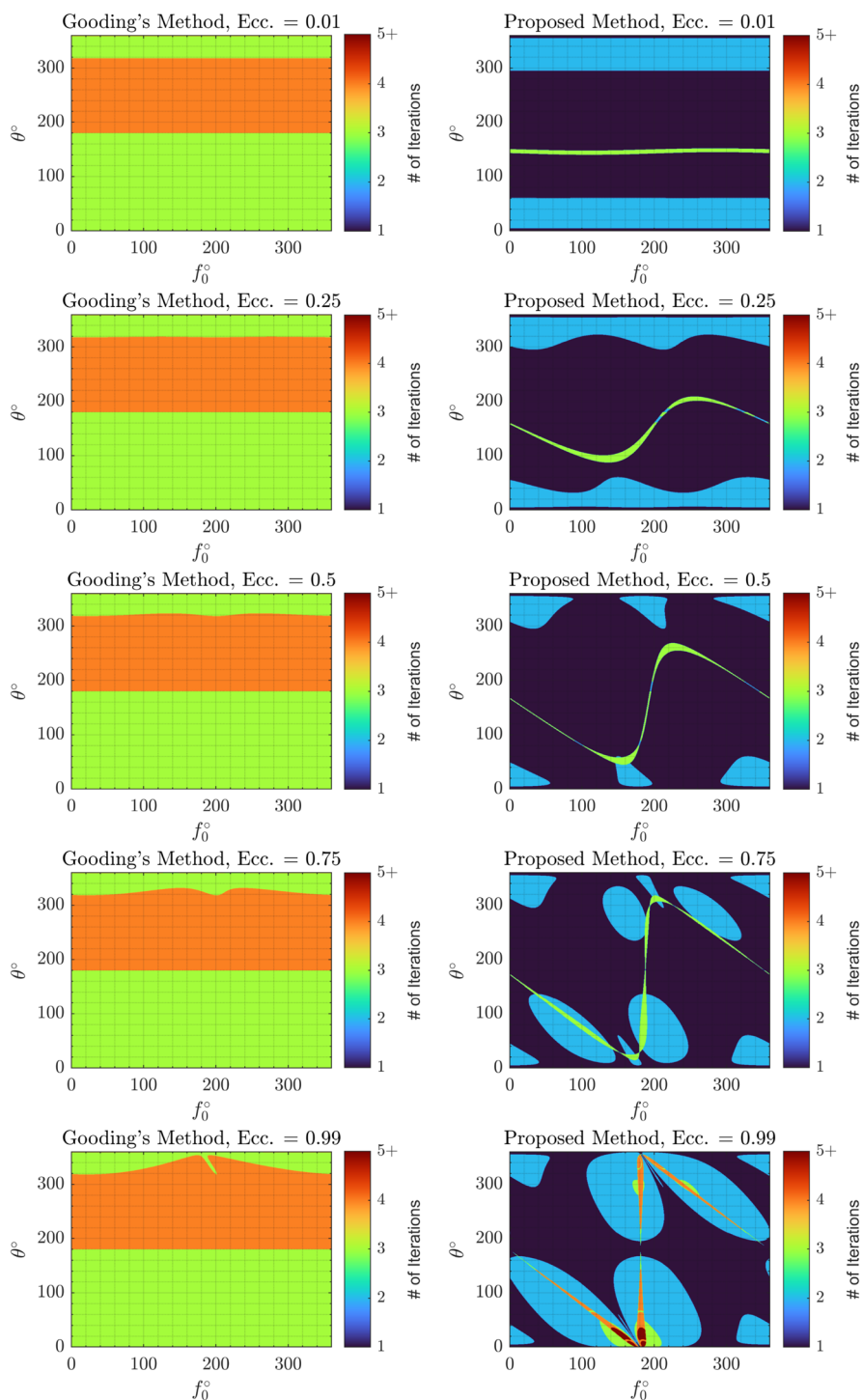


Fig. 9 Number of iterations for the $N = 1$ cases to converge to a value of x_m

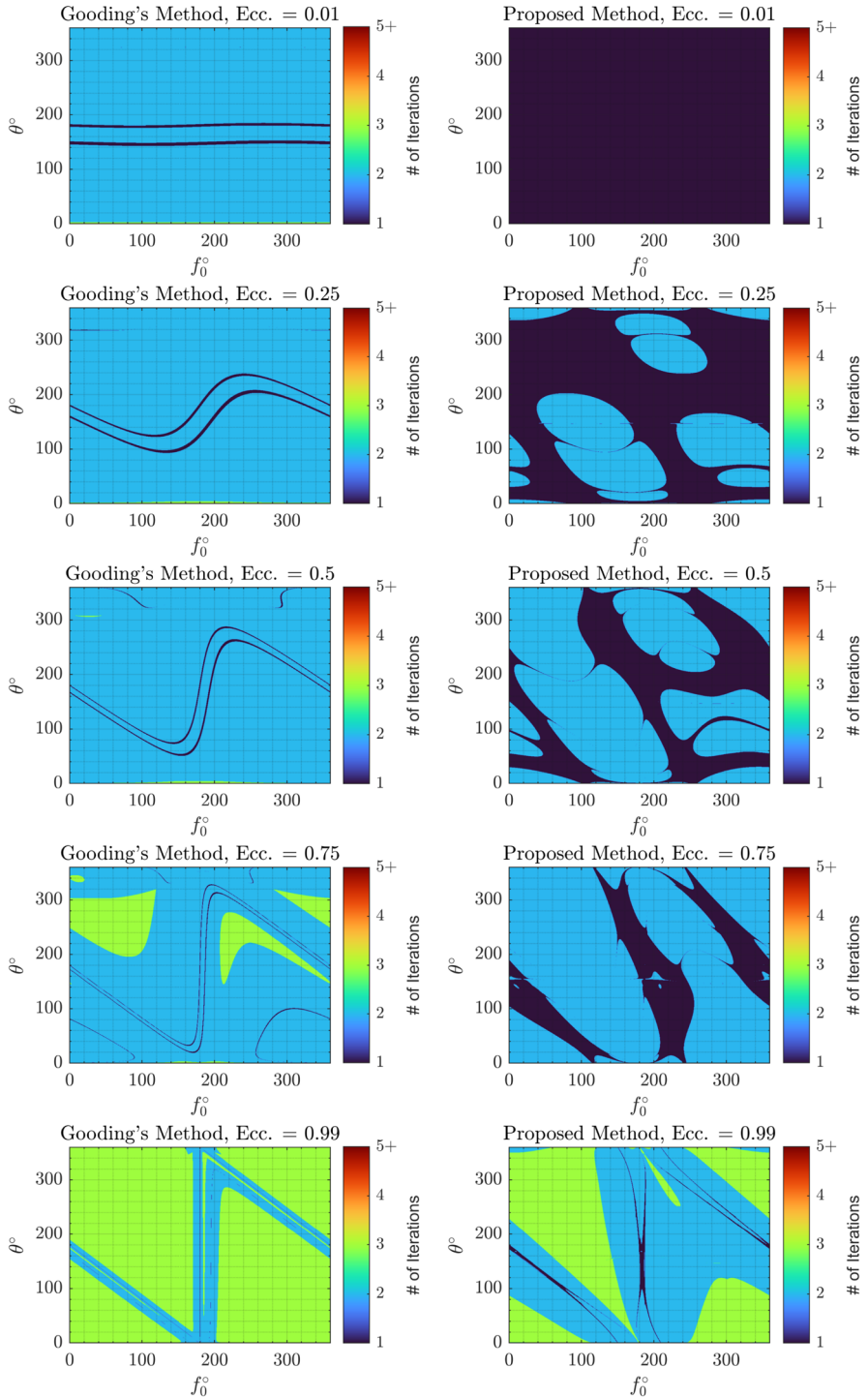


Fig. 10 Number of iterations for the $N = 1$ cases to reach 10-digit velocity accuracy

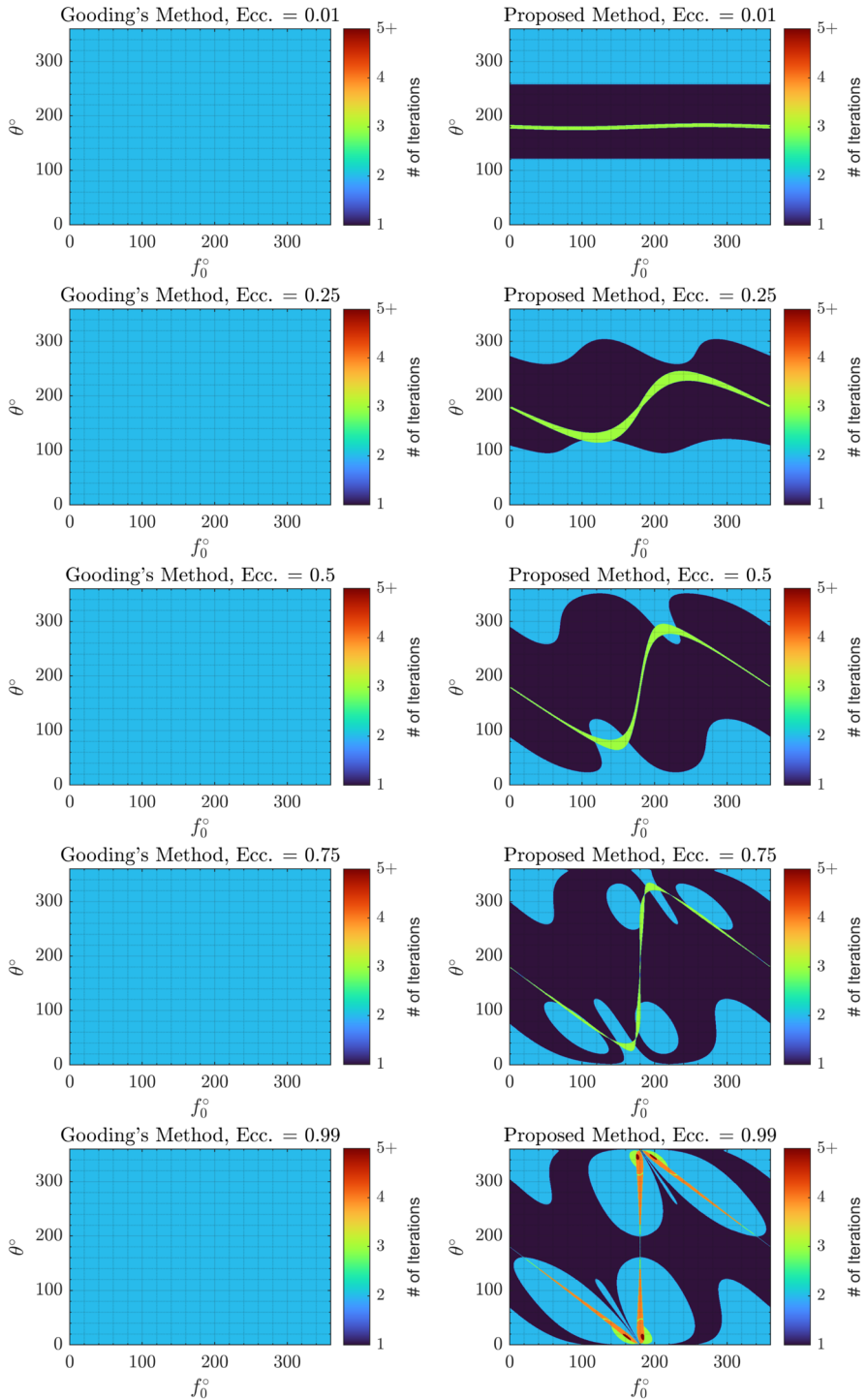


Fig. 11 Number of iterations for the $N = 1000$ cases to converge to a value of x_m

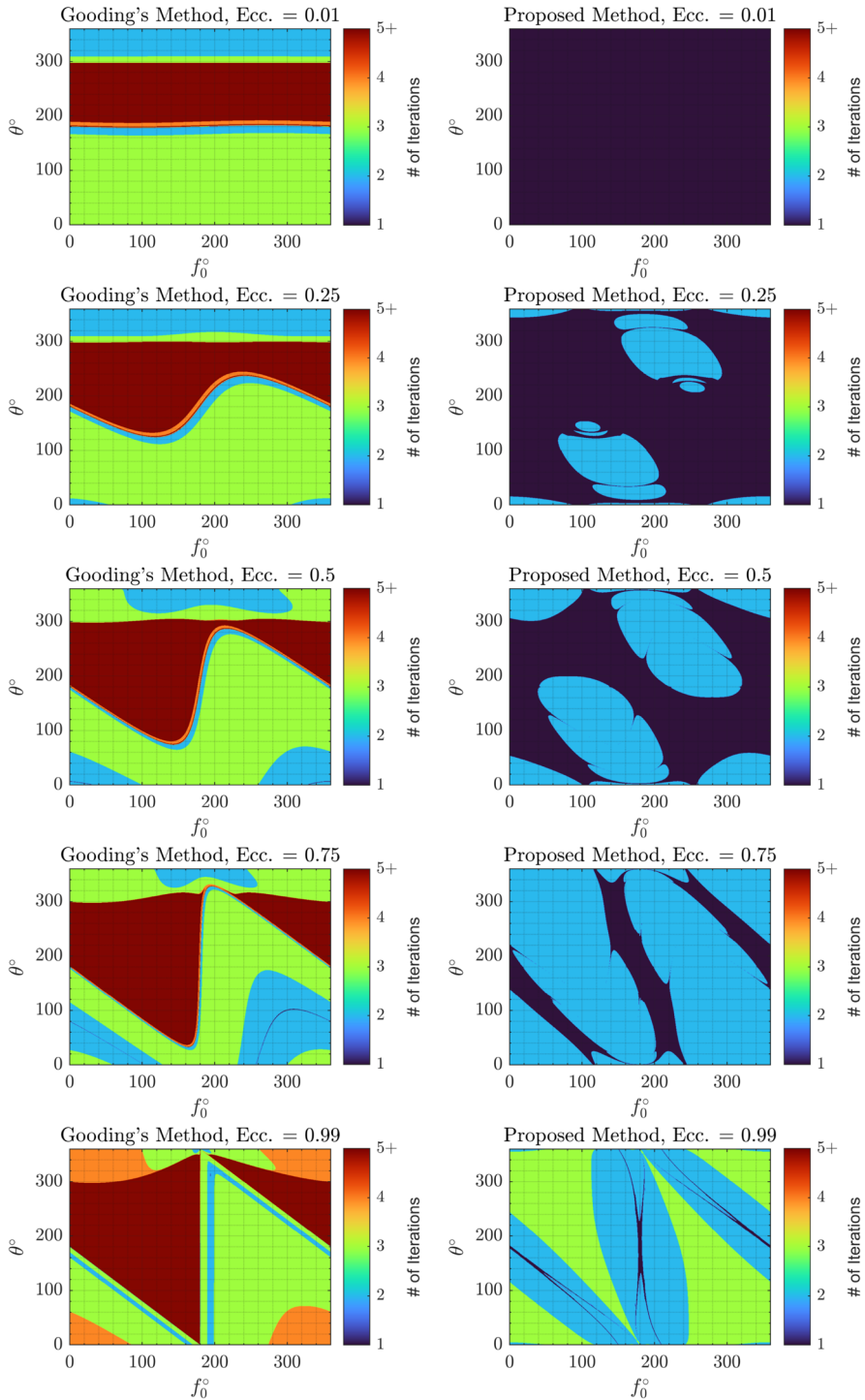


Fig. 12 Number of iterations for the $N = 1000$ cases to reach 10-digit velocity accuracy

Table 3 Number of multi-revolution failed cases; Gooding's method

(307,074,240 total cases)	N=1	N=10	N=100	N=1,000	N=10,000	N=100,000
Failed to meet velocity accuracy	49	31	54	75	69	54
Converged to wrong solution	0	0	0	0	2054	2475

Table 4 Number of multi-revolution failed cases; proposed method

(307,074,240 total cases)	N=1	N=10	N=100	N=1,000	N=10,000	N=100,000
Failed to meet velocity accuracy	32	20	33	44	35	29
Converged to wrong solution	0	0	0	0	0	0

but neither converged to the wrong solution for any of the ~ 50 million cases tested. When looking at Fig. 9, it is quickly apparent from the color contours that the proposed method converged in fewer iterations for almost all geometries. The geometries where the proposed method took more iterations to converge generally corresponded to solves where the desired transfer time was close to the minimum, resulting in a green “S” band that appears in the contour plots. When looking at the average number of iterations to solve for x_m , the left side of Figs. 13 and 14, the proposed method was observed to solve in ~ 2 iterations faster for all eccentricities for $N = 1$. When looking at Fig. 10, it was quickly apparent that the number of iterations when solving for x was less for the proposed method, with many geometries solving in one iteration even up to 0.75 eccentricity. Figures 15 and 16 support this, as the average solver iterations for $N = 1$ were less for the proposed method across all eccentricities. Most notably, in Fig. 16, is how the proposed solver converged, on average, in one iteration for eccentricities of ~ 0.1 or less.

With regard to Fig. 12, Gooding's method began to struggle when solving for x due to the increase in revolutions. While Gooding's method initial guess for x_m became more accurate as N increased, Fig. 11, the initial guesses for x became noticeably worse, Fig. 12, taking five iterations to achieve the desired velocity accuracy in many different geometries across all eccentricities. In contrast, the proposed method was almost unaffected by the increase from $N = 1$ to $N = 1000$, as the initial guesses for x_m and x remained accurate. These takeaways are supported by Figs. 13–16, where the revolutions are increased from 1 to 100,000. Note that certain profiles overlap, making it difficult to distinguish profiles. To determine where an overlapped profile appears, the rule of thumb to use is that the profile is under the next largest N that is visible. For example, in Fig. 13, the 99 percentile $N = 10$ profile is underneath the $N = 100$ profile and the $N = 1000$, 10000 profiles are underneath the $N = 100000$ profile. While Gooding's iteration count for x_m decreased to an average of 2 for $N \geq 1000$, the average number of iterations for x continuously increased with N , up to an average greater than four iterations. For the proposed method, the number of iterations required for x_m increased slightly with N but leveled out at $N = 1000$. When solving for x , the average iteration count remained unchanged for $N \geq 10$, maintaining the one iteration average convergence for eccentricities below 0.1, even up to $N = 100,000$. When considering the 99 percentile, the proposed method was 2 or less for all revolutions with eccentricities below 0.95. This reveals that, even for the ultra-revolution regime, most geometries will achieve 10 digits of velocity accuracy in two iterations or less when using the proposed method. In contrast, Gooding's method can be seen to increase up to seven iterations for the 99 percentile,

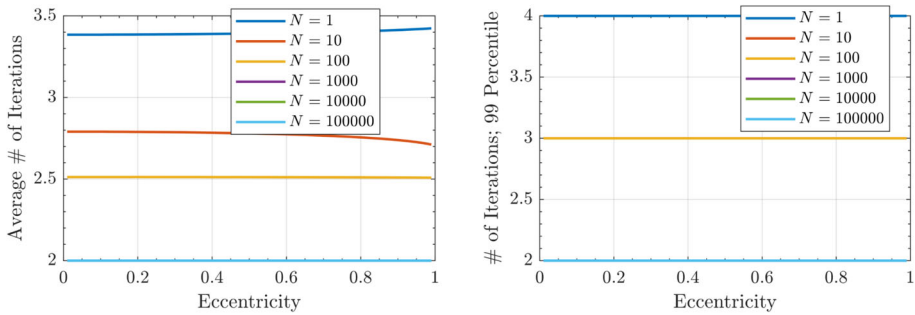


Fig. 13 Average and 99 percentile of minimum transfer time solution solver iterations for multi-revolution cases; Gooding's method

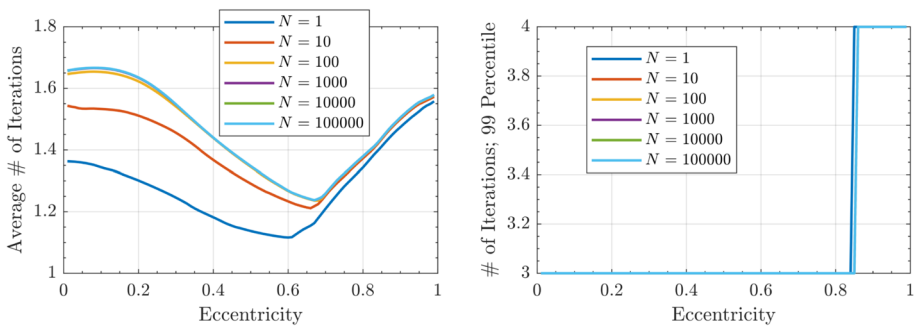


Fig. 14 Average and 99 percentile of minimum transfer time solution solver iterations for multi-revolution cases; proposed method

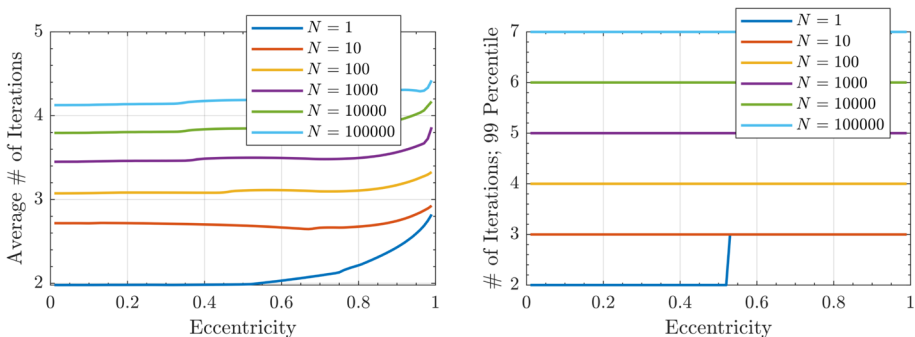


Fig. 15 Average and 99 percentile of desired solution solver iterations for multi-revolution cases; Gooding's method

indicating many cases that require quite a few iterations to solve. Finally, when looking at the number of failed cases, Tables 3 and 4, Gooding's method began to have instances of converging to the wrong solution at $N = 10,000$, while none of the solves of the proposed method converged to the wrong solution over all multi-revolution cases tested.

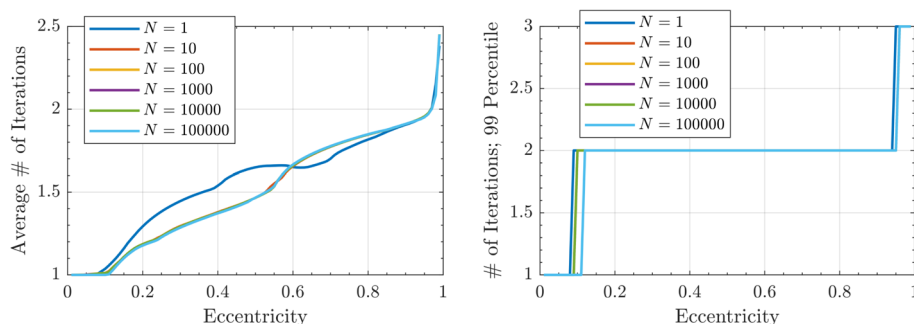


Fig. 16 Average and 99 percentile of desired solution solver iterations for multi-revolution cases; proposed method

5.2 Solver speed and velocity error

To further compare the performance of the proposed solver and Gooding's, each solver's computation time and final velocity error were determined across all transfer geometries. As a reminder, prior solvers such as those by Izzo (2015); Arora and Russell (2013), and Russell (2019) have reported speed improvements over Gooding's method ranging from 20% to 68%, depending on the specific formulation and geometry being solved. For this analysis, the convergence criteria of Eq. 69 with $\epsilon = 10^{-10}$ were employed for the proposed method. Gooding's method was evaluated using its standard three-iteration implementation. While limiting the method to three iterations can lead to large velocity errors, particularly at high revolution counts where the method tends to struggle, permitting additional iterations increases computational time. For the speed and accuracy comparison, the authors felt that retaining the three-iteration configuration provides the most appropriate and representative benchmark, as it reflects how the method is typically deployed in practice. Note that computation speed depends significantly on the machine employed, the programming language/compiler used, and the optimality of the code written for a language. An entirely separate paper could be written to properly investigate and optimize Lambert solver speeds, so an extensive investigation of optimal implementations lies beyond the scope of this work. Thus, the authors wish to emphasize that these findings should be interpreted with caution, as they might not remain consistent across different implementations.

The average computation speed and average final velocity error results are listed in Tables 5 and 6, respectively. In both tables, the ratio of Gooding's value over the proposed is also listed. For hyperbolic cases, it was found that Gooding's method solved hyperbolic cases noticeably faster (about 50%), despite the similarities in the number of iterations to reach 10 digits of velocity accuracy in the previous analysis. The notable time reduction is due to the increased computational burden of tracking imaginary values when solving hyperbolic cases with the proposed method. Despite the computation time increase, the proposed method was seen to achieve a nearly two-order-of-magnitude velocity error decrease over Gooding's. This is because Gooding's method has trouble handling cases of small transfer angles when the eccentricity is near 1. Regarding elliptic cases, the proposed method was seen to solve faster than Gooding's, on average, for all numbers of revolutions tested. The speed-ups varied across revolution count, between 27 and 49%, with the speed-ups decreasing slightly as the number of revolutions increased beyond 1. However, it is important to note the difference in the average velocity error. Gooding's method performed significantly worse than the proposed

Table 5 Average computation time for Gooding's and the proposed method

	Gooding's (micro-sec)	Proposed (micro-sec)	Ratio (Gooding/Proposed)
Hyperbolic	3.21	5.60	0.57
N=0 Elliptic	3.00	2.37	1.27
N=1 Elliptic	5.69	3.57	1.49
N=10 Elliptic	5.61	4.03	1.39
N=100 Elliptic	5.46	3.97	1.37
N=1,000 Elliptic	5.37	4.13	1.30
N=10,000 Elliptic	5.40	4.17	1.30
N=100,000 Elliptic	5.38	4.13	1.30

Table 6 Average velocity error for Gooding's and the proposed method

	Gooding's (km/s)	Proposed (km/s)	Ratio (Gooding/Proposed)
Hyperbolic	5.22×10^{-11}	5.56×10^{-13}	9.40×10^1
N=0 Elliptic	3.37×10^{-5}	4.90×10^{-13}	7.74×10^7
N=1 Elliptic	3.57×10^{-13}	2.71×10^{-13}	1.32×10^0
N=10 Elliptic	1.94×10^{-11}	2.66×10^{-13}	7.29×10^1
N=100 Elliptic	1.65×10^{-5}	2.29×10^{-13}	7.18×10^7
N=1,000 Elliptic	2.24×10^{-2}	2.22×10^{-13}	1.01×10^{11}
N=10,000 Elliptic	2.76×10^{-1}	2.19×10^{-13}	1.26×10^{12}
N=100,000 Elliptic	6.33×10^{-1}	2.13×10^{-13}	2.97×10^{12}

method when it came to final velocity error, with the difference in error being over 10 orders of magnitude greater at revolutions greater than 1000. This was unsurprising, as the previous analysis clearly showed that Gooding's method would need, on average, more than three iterations to achieve a reasonable velocity accuracy at large numbers of revolutions. Additionally, the large error in Gooding's method for the single-revolution case is explained by the fact that Gooding's method had trouble converging for small transfer angles near eccentricity of 1. Thus, for elliptic cases, the proposed method was seen to solve faster than Gooding's method and achieve a significantly higher degree of velocity accuracy.

6 Concluding remarks

This work introduces a universal approach to solving the multiple-revolution Lambert problem, capable of solving for orbital transfers across elliptic and hyperbolic regimes using a single formulation. By leveraging the Sundman transform and matrix exponential methods, the solver simplifies the process of evaluating the transfer time function. A major contribution of this solver is that it only requires exponential function evaluations for the transfer time instead of trigonometric functions, continued fractions, or power series. This eliminates the need for base/inverse trigonometric calculations during the iteration process and removes the need to swap between standard and hyperbolic trigonometric functions. These simplifications reduce the complexity of the Lambert solver algorithm and increase the ease of

implementation. Another advantage of the method is the universal variable selected, as it is an easy parameter to both visualize and understand (change in eccentric/hyperbolic anomaly for elliptic/hyperbolic trajectories, respectively). The universal variable is easily bounded in this solver, and the distinct multi-revolution term in the transfer time function is removed. The removal of the multi-revolution term reduces partial derivative complexity and improves the numerical stability of the transfer time equation for large revolution counts. Also, the universal variable's relation to the transfer angle allows for the transfer angle to be leveraged for accurate initial guess schemes even at 100,000 revolutions.

The solver's efficiency was validated through numerical simulations, where the solver was compared against what is widely considered the benchmark, Gooding's Lambert solver. Over 350,000,000 cases were tested, varied across the number of revolutions, conics, eccentricities, and transfer angles. For the first analysis, the number of iterations to reach 10 digits of velocity accuracy was compared. With regard to single-revolution solutions, Gooding's method and the proposed method performed similarly in the hyperbolic regime. In the elliptic regime, the proposed method achieved 10 digits of velocity accuracy, on average, in fewer iterations than Gooding's across all eccentricities tested. Gooding's method was also seen to have trouble reaching the desired velocity accuracy when the eccentricity was near a value of 1 and the transfer angle was low. For the multi-revolution solutions, cases were tested up to 100,000 revolutions. Gooding's method performed well at low numbers of revolutions, but as the number of revolutions increased, it began degrading to the point where it began converging to incorrect solutions. With regard to the proposed method, the solver was seen to remain accurate even at 100,000 revolutions, reaching 10 digits of velocity accuracy in, on average, 1 iteration for eccentricities less than 0.1 and two iterations for eccentricities less than 0.95, regardless of revolution count.

When investigating computation speeds and final velocity error, it was found that Gooding's method solved hyperbolic cases faster in part because the proposed method tracks complex-valued intermediate quantities during hyperbolic iterations. While the final outputs remain real, the solver requires complex arithmetic in a localized portion of the computation. However, the proposed approach attained better velocity accuracy due to Gooding's method struggling when the eccentricity is near 1 and the transfer angles are small. For elliptical orbits, the proposed method is solved, on average, between 27 and 49% faster than Gooding's, depending on the number of revolutions, aligning with speed-ups seen by Izzo (2015); Arora and Russell (2013), and Russell (2019). The proposed method also achieved markedly lower average velocity errors (up to 12 orders of magnitude), particularly when considering large revolution counts. Despite the slower solving of hyperbolic cases, the improved solving speeds of elliptic solutions and lower overall velocity error make the proposed method a compelling and efficient alternative for solving Lambert's problem in modern applications.

Appendix A

Transfer time partial derivatives

Appendix A serves to provide the first, second, and third-order derivatives of the transfer time equation and α . The first (T'), second (T''), and third derivatives (T''') of T with respect to x are given by:

$$\alpha T' = (mU_0^* + U_2)\sqrt{\alpha} - \left(\frac{1}{2}T + U_3\right)\alpha' \quad (\text{A1})$$

$$\alpha T'' = U_1 - \frac{1}{2}m\alpha U_1^* + \alpha' \left(\frac{2mU_0^*}{\sqrt{\alpha}} - 3T' \right) - \frac{3T}{4\alpha}(\alpha')^2 - \left(\frac{1}{2}T + U_3 \right) \quad (\text{A2})$$

$$\begin{aligned} \alpha T''' = & \frac{U_0}{\sqrt{\alpha}} - \frac{1}{2}m\sqrt{\alpha}U_0^* - \alpha' \left[\frac{9}{2}T'' + \frac{3}{2}mU_1^* + \alpha' \left(\frac{9}{4}T' - \frac{3T\alpha'}{8\alpha^2} \right) \right] \\ & + \alpha'' \left[\frac{2mU_0^*}{\sqrt{\alpha}} - \frac{U_2}{\sqrt{\alpha}} - \frac{7}{2}T' + \alpha' \left(\frac{U_3}{\alpha} - \frac{7T}{4\alpha} \right) \right] - \alpha''' \left(\frac{1}{2}T + U_3 \right) \end{aligned} \quad (\text{A3})$$

The first (α'), second (α''), and third derivatives (α''') of α with respect to x are given by:

$$\alpha' = \frac{\sqrt{\alpha}(2U_0^* - m\alpha)}{2U_1^*} \quad (\text{A4})$$

$$\alpha'' = -\frac{\alpha}{2} + \alpha' \left[\frac{U_0^* - 2m\alpha}{2\sqrt{\alpha}U_1^*} \right] \quad (\text{A5})$$

$$\alpha''' = \alpha' \left[\frac{2m^2\alpha - mU_0^*}{4U_2} - \alpha' \frac{U_0^* + 2\alpha m}{4\alpha^{\frac{3}{2}}U_1^*} - \frac{3}{4} \right] + \alpha'' \frac{U_0^* - 2m\alpha}{2\sqrt{\alpha}U_1^*} \quad (\text{A6})$$

Appendix B

Evaluation of χ for $\alpha = 0$

Appendix B serves to show the evaluation χ when $\alpha = 0$. Start by substituting Eq. 38 into (27) and rearranging for χ .

$$\chi = \frac{x}{\sqrt{\alpha}} = \frac{x\sqrt{r_f + r_0 - 2mU_0^*}}{\sqrt{1 - U_0}} \quad (\text{B7})$$

Next, the definition of U_2 is substituted in, Eq. 18, and the definition of x , Eq. 27.

$$\chi = \frac{\chi\sqrt{\alpha}\sqrt{r_f + r_0 - 2mU_0^*}}{\sqrt{\alpha}U_2} \quad (\text{B8})$$

The $\sqrt{\alpha}$ cancels each other

$$\chi = \frac{\chi\sqrt{r_f + r_0 - 2mU_0^*}}{\sqrt{U_2}} \quad (\text{B9})$$

and the values of the universal functions evaluated when $\alpha = 0$ are substituted in.

$$\chi(\alpha = 0) = \frac{\chi\sqrt{r_f + r_0 - 2m(1)}}{\sqrt{\frac{\chi^2}{2}}} \quad (\text{B10})$$

The χ values cancel each other, yielding:

$$\chi(\alpha = 0) = \sqrt{2(r_f + r_0 - 2m)} \quad (\text{B11})$$

7 Evaluation of universal functions by trigonometric functions

Appendix C serves to provide the expressions of the universal functions constructed using trigonometric functions. Using the identities of

$$\cos x = \frac{1}{2}(e^{jx} + e^{-jx}) \quad (\text{C11})$$

and

$$\sin x = j\frac{1}{2}(e^{jx} - e^{-jx}) \quad (\text{C12})$$

in combination with the definitions of the universal functions, Eqs. 16-19 and 22-23, expressions for the universal functions are easily constructed.

$$U_0(\chi; \alpha) = \cos \sqrt{\alpha} \chi \quad (\text{C13})$$

$$U_1(\chi; \alpha) = \frac{\sin \sqrt{\alpha} \chi}{\sqrt{\alpha}} \quad (\text{C14})$$

$$U_2(\chi; \alpha) = \frac{1 - \cos \sqrt{\alpha} \chi}{\alpha} \quad (\text{C15})$$

$$U_3(\chi; \alpha) = \frac{\sqrt{\alpha} \chi - \sin \sqrt{\alpha} \chi}{\alpha \sqrt{\alpha}} \quad (\text{C16})$$

$$U_0^*(\chi; \alpha) = \cos \frac{1}{2} \sqrt{\alpha} \chi \quad (\text{C17})$$

$$U_1^*(\chi; \alpha) = \frac{\sin \frac{1}{2} \sqrt{\alpha} \chi}{\sqrt{\alpha}} \quad (\text{C18})$$

Acknowledgements The authors would like to acknowledge Waylon Lee for his insights into debugging and refining the algorithm implementation.

Funding The author received no financial support, grants, or other funding for the research, authorship, or publication of this manuscript.

Declarations

Conflict of interests The authors confirm that the algorithmic approach presented in this manuscript is unique to this study and has not been published elsewhere. In the interest of advancing the field and promoting reproducible research, the MATLAB code used to produce the results can be found using the following link: <https://www.mathworks.com/matlabcentral/fileexchange/181322-a-universal-multi-revolution-lambert-solver>.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Arora, N., Russell, R.P.: A fast and robust multiple-revolution lambert algorithm using a cosine transformation. In: Proceedings of the AAS/AIAA Astrodynamics Specialist Conference, AAS Paper pp. 13–728 (2013)
- Battin, R.H.: An Introduction to the Mathematics and Methods of Astrodynamics. American Institute of Aeronautics and Astronautics, Reston, VA (1999). <https://doi.org/10.2514/4.861543>
- Bate, R.R., Mueller, D.D., White, J.E., Saylor, W.W.: Fundamentals of Astrodynamics. Dover Publications, New York (2020)
- Boltz, F.W.: Second-order p-iterative solution of the lambert/gauss problem. *J. Astronaut. Sci.* **32**(4), 475–485 (1984)
- Battin, R.H., Vaughan, R.M.: An elegant lambert algorithm. *J. Guid. Control. Dyn.* **7**(6), 662–670 (1984). <https://doi.org/10.2514/3.19910>
- Escobal, P.R.: Methods of Orbit Determination. John Wiley & Sons, New York (1965)
- Gooding, R.H.: A procedure for the solution of lambert's orbital boundary-value problem. *Celest. Mech. Dyn. Astron.* **48**(2), 145–165 (1990). <https://doi.org/10.1007/BF00049511>
- Izzo, D.: Revisiting lambert's problem. *Celest. Mech. Dyn. Astron.* **121**(1), 1–15 (2015). <https://doi.org/10.1007/s10569-014-9587-y>
- Klumpp, A.: Performance comparison of lambert and kepler algorithms. Interoffice Memorandum 314.1-0426-ARK, Jet Propulsion Laboratory, Pasadena, CA (1991)
- Lancaster, E.R., Blanchard, R.C.: A unified form of lambert's theorem. NASA Technical Note NASA-TN-D-5368, National Aeronautics and Space Administration, Washington, D.C. (1969). <https://ntrs.nasa.gov/api/citations/19690027552/downloads/19690027552.pdf>
- Majji, M.: Regular celestial mechanics and the matrix exponential. *J. Guid. Control. Dyn.* **43**(11), 2127–2132 (2020). <https://doi.org/10.2514/1.G005237>
- Muller, J.-M., Brunie, N., Dinechin, F., Jeannerod, C.-P., Joldes, M., Lefèvre, V., Melquiond, G., Revol, N., Torres, S.: Handbook of Floating-Point Arithmetic. Birkhäuser, Switzerland (2018). <https://doi.org/10.1007/978-3-319-76526-6>
- Muller, J.-M.: Elementary Functions: Algorithms and Implementation, 3rd edn. Birkhäuser, Boston, MA (2016). <https://doi.org/10.1007/978-1-4899-7983-4>
- Moler, C., Van Loan, C.: Nineteen dubious ways to compute the exponential of a matrix. *SIAM Rev.* **20**(4), 801–836 (1978). <https://doi.org/10.1137/1020098>
- Negrete, A., Abdelkhalik, O.O.: Exact solution to lambert's problem using contour integrals. *J. Guid. Control. Dyn.* **48**(4), 771–780 (2025). <https://doi.org/10.2514/1.G008499>
- Peterson, G., Campbell, E.T., Balbas, J., Ivy, S., Merkurjev, E., Hall, T., Rodriguez, P.: Relative performance of lambert solvers 1: Zero-revolution methods. *Adv. Astronaut. Sci.* **136**(1), 1495–1510 (2010)
- Russell, R.P.: On the solution to every lambert problem. *Celest. Mech. Dyn. Astron.* **131**(11), 50 (2019). <https://doi.org/10.1007/s10569-019-9927-z>
- Russell, R.P.: Complete lambert solver including second-order sensitivities. *J. Guid. Control. Dyn.* **45**(2), 196–212 (2022). <https://doi.org/10.2514/1.G006089>
- Shen, H., Tsiotras, P.: Using battin's method to obtain multiple-revolution lambert's solutions. In: AAS/AIAA Astrodynamics Specialist Conference, Big Sky, MT (2003). AAS Paper 03-568
- Stark, H.M.: Optimum trajectories between two terminals in space. *ARS J.* **31**, 261–263 (1961)
- Sundman, K.F.: Mémoire sur le problème des trois corps. *Acta Math.* **36**(1), 105–179 (1913). <https://doi.org/10.1007/BF02422379>
- Vallado, D.A.: Fundamentals of Astrodynamics and Applications, vol. 12. Springer, California (2001)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.